

به نام خدا



دانشگاه شهید بهشتی

دانشکده مهندسی برق (کنترل و قدرت)

دستور کار آزمایشگاه پردازش سیگنال های دیجیتال

تهیه کننده: دکتر محمد جوادیان

فهرست

آزمایش ۱: سیگنال‌های زمان گسسته در حوزه زمان	۴
بخش ۱: آشنایی با دستورات متلب	۴
بخش ۲: تابع ضربه و تابع پله‌ی واحد، سینوسی و تصادفی	۶
آزمایش ۲: تحلیل سیگنال‌های زمان گسسته در حوزه زمان	۱۱
بخش ۱: بررسی فیلتر میانگین یا هموار ساز	۱۱
بخش ۲: بررسی یک سیستم غیرخطی ساده	۱۲
بخش ۳: بررسی خاصیت تغییرپذیری یا تغییرناپذیری با زمان	۱۳
بخش ۴: محاسبه پاسخ ضربه	۱۳
بخش ۵: سری کردن سیستم‌ها	۱۴
بخش ۶: کانولوشن	۱۴
بخش ۷: بررسی پایداری سیستم‌ها	۱۵
آزمایش ۳: سیگنال‌های زمان گسسته در حوزه فرکانس	۱۶
بخش ۲: شیف‌ت زمانی تبدیل فوریه گسسته	۱۸
بخش ۳: شیف‌ت فرکانسی تبدیل فوریه گسسته	۱۸
بخش ۴: ویژگی کانولوشن و ضرب تبدیل فوریه گسسته	۱۹
بخش ۵: ویژگی معکوس کردن زمان	۱۹
بخش ۶: تبدیل فوریه سریع FFT	۱۹
بخش ۷: ویژگی‌های شیف‌ت و کانولوشن چرخشی fft	۲۰
آزمایش ۴: تحلیل سیگنال‌های زمان گسسته در حوزه فرکانس	۲۲
بخش ۱: رسم قطب‌ها و صفرهای تبدیل Z سیستم	۲۲
بخش ۲: بررسی ویژگی سیستم‌ها، انواع فیلترها	۲۳
بخش ۳: محاسبه بهره تابع تبدیل	۲۴
بخش ۴: بررسی پایداری فیلترها	۲۴
آزمایش ۵: طراحی فیلترهای دیجیتال FIR	۲۶

بخش ۱: طراحی فیلتر FIR	۲۶
بخش ۲: استفاده از تولباکس filter Designer با fdatool	۲۹
آزمایش ۶: طراحی فیلترهای دیجیتال IIR	۳۱
بخش ۱: تعیین درجه فیلتر IIR	۳۱
بخش ۲: طراحی فیلتر IIR	۳۲
آزمایش ۷: نمونه برداری از سیگنال پیوسته، افزایش و کاهش نرخ نمونه برداری	۳۴
بخش اول: نمونه برداری و بررسی اثرات	۳۴
بخش ۲: بررسی افزایش نرخ نمونه برداری (upsampling) و کاهش نرخ نمونه برداری (Downsampling)	۳۵
آزمایش ۸: پردازش تصاویر دیجیتال	۳۷
بخش ۱: خواندن تصویر و مشاهده هیستوگرام	۳۷
بخش ۲: اعمال تبدیل روی تصاویر	۳۸
بخش ۳: اعمال فیلترهای مختلف بر روی تصاویر	۳۸
آزمایش ۹: پردازش سیگنال های صوتی	۴۰
آزمایش ۱۰: پردازش سیگنال های تصادفی EEG با هوش مصنوعی	۴۸
آزمایش ۱۱: پیاده سازی برخی مدولاسیون های مخابرات دیجیتال	۵۱
بخش ۱: مدولاسیون ASK	۵۱
بخش ۲: مدولاسیون BPSK	۵۲
بخش ۳: مدولاسیون FSK	۵۲

آزمایش ۱: سیگنال‌های زمان گسسته در حوزه زمان

در این آزمایش، با برخی دستورات MATLAB آشنا شده، سپس چند سیگنال پایه زمان-گسسته را پیاده سازی کرده و عملیات ابتدایی را روی این سیگنال‌ها انجام می‌دهیم.

بخش ۱: آشنایی با دستورات متلب

دستورات اولیه:

clc, clear, close all

disp('Result is = ')

w = input('Enter a number = ')

ans, Nan, pi, j, I, inf, eps, nargin, nargin

real, imag, abs, angle, sum, sin, cos, tan, sinh,

asin, acish, exp, round, log10,

Numeric, logical, char, cell, ...

Double, uint8, uint16, uint32, int8. Int16, ...

axis, grid, legend, plot, stem, stairs, title,

xlabel, ylabel, subplot, figure, clf, sawtooth,

square, ...

ones, eye, zeros, rand, randn, randperm, ...

نمایش نتیجه

دریافت ورودی از کاربر

متغیرهای خاص متلب

برخی از توابع مهم

انواع متغیرها

ابزار ترسیم

ماتریس‌های خاص

فعالیت ۱: برنامه ای بنویسید که یک عدد مختلط از کاربر دریافت کرده، اندازه دامنه و زاویه عبارت زیر را محاسبه کرده و نتیجه را نمایش دهد.

$$y = x^2 + 3x + 10$$

فعالیت ۲: با استفاده از اپراتور : اعداد زوج دو رقمی را تولید کنید. با دستور linspace اعداد زوج یک رقمی را تولید کنید. سپس دو آرایه ایجاد شده را با هم concat کنید.

فعالیت ۳: برنامه ای بنویسید که عدد صحیح n را از کاربر بگیرد، سپس بردار x را بین $2n\pi$ و $-2n\pi$ را ایجاد کرده و y را محاسبه کنید.

$$y = |\sin(x)| + 5x^3 + 20$$

فعالیت ۴: برنامه ای بنویسید که ماتریس زیر را بدون وارد کردن مستقیم اعداد ایجاد کند:

$$\begin{bmatrix} 1 & \dots & 10 \\ \vdots & \ddots & \vdots \\ 1 & \dots & 10 \end{bmatrix}_{10 \times 10}$$

فعالیت ۵: برنامه‌ای بنویسید که ماتریس دو ستونی از کاربر دریافت کند، به نحوی که ستون اول نمرات ۵ درس یک ترم دانشجو باشد و ستون دوم تعداد واحد هر درس، سپس عملیات زیر را بر روی آن انجام دهد:

(۱) محاسبه تعداد واحدها

(۲) محاسبه معدل

(۳) نمایش نتیجه با پیام مناسب

فعالیت ۶: آشنایی با دستورات for, if, while:

```
x = [];
for n = 1:10
    x(n) = sin(n * pi/10)
end
```

```
Ages = input('Ages = ');
k = length(Ages);
for i = 1:k
    if Ages(i) < 17
        disp('Teen')
    elseif Ages(i) > 17 && Ages(i) < 40
        disp('Young')
    elseif Ages(i) > 40 && Ages(i) < 60
        disp('Middle - aged')
    end
end
end
```

فعالیت ۷: تابعی بنویسید که نمرات ۱۰ دانشجوی یکسان را به همراه شماره دانشجویی از ۵ استاد بعنوان آرگومان ورودی دریافت کرده و میانگین نمرات هر درس، و معدل هر دانشجو را بعنوان خروجی تولید کرده و نمایش دهد. راهنمایی: ایجاد توابع در متلب:

```
function [out1,out2,out3,...] = Name(input1,input2,inut3,...)
    %comments
    Rules
end
```

فعالیت ۸: یک ماتریس ۳*۳ با درایه‌های تصادفی ایجاد کرده، سپس با استفاده از توابع det, inv, pinv, trace سایر توابع کار با ماتریس نتیجه را مشاهده کنید.

بخش ۲: تابع ضربه و تابع پله‌ی واحد، سینوسی و تصادفی

فعالیت ۹: بردارهای زیر را ایجاد کرده، انرژی آنها را محاسبه کرده و با استفاده از ابزار ترسیم در متلب، نتیجه را ترسیم کنید.

توجه: برای ترسیم توابع زمان گسسته عموماً از دستور stem استفاده میشود.

توجه: در صورتیکه بخواهیم در سیگنال با طول N، تاخیر به اندازه M سمپل ایجاد کنیم (M<N)، میتوانیم به مقدار تاخیر، صفر به ابتدای سیگنال اضافه کنیم.

ابزار ترسیم:

axis, grid, legend, plot, stem, title, xlabel, ylabel, subplot, figure, ...

$$Energy = \sum_{n=0}^M |x[n]|^2 \quad (x \text{ سیگنال حقیقی}) \Rightarrow \text{sum}(X.*X)$$

$$\delta = [1 \text{ zeros}(1, N - 1)]$$

$$U = [\text{ones}(1, N)]$$

$$\delta = [\text{zeros}(1, M) \quad 1 \quad \text{zeros}(1, N - M - 1)]$$

$$X = A * \exp(n * c)$$

$$X = A * b.^n$$

تابع ضربه

تابع پله واحد

تابع ضربه تاخیر یافته به اندازه M نمونه

تابع نمایی

تمرین ۵: کدام پارامتر، نرخ رشد و یا نزول این رشته را کنترل می‌کند؟ با کدام پارامتر می‌توان دامنه‌ی این سیگنال را تغییر داد؟

تمرین ۸: نرخ رشد و نزول و دامنه‌ی این سیگنال توسط چه متغیرهایی تغییر می‌کند؟

تمرین ۹: تفاوت بین عملگر $\wedge$ با $\wedge$ در چیست؟

تمرین ۱۰: پارامتر a را به عددی کمتر از یک تغییر دهید، چه تغییری در سیگنال حاصل می‌شود؟ برنامه را به ازای $a=0.9$ و $k=20$ اجرا کنید.

تمرین ۱۱: طول سیگنال چند است و چگونه می‌توان آن را تغییر داد؟

تمرین ۱۲: انرژی سیگنال را یک بار در حالت اولیه‌ی تمرین P1-3 و دفعه‌ی دیگر بعد از اعمال تغییرات تمرین ۱۰ محاسبه کنید. چه تفاوتی در انرژی سیگنال ایجاد می‌شود؟ چرا؟
نکته: برای محاسبه‌ی انرژی سیگنال حقیقی گسسته S در متلب می‌توان از عبارت $\text{sum}(s.*s)$ کمک گرفت.

$$X = A * \cos(2 * \pi * f * n + \varphi) \quad \text{و} \quad \text{دوره تناوب} = \frac{1}{f}$$

فعالیت ۱۰: یک سیگنال کسینوسی یک بار با فرکانس 0.9 و یکبار با فرکانس 1.1 ایجاد کرده و با فاز ۹۰ درجه و در یک تصویر واحد همراه با legend ترسیم کرده و توان متوسط این سیگنال‌ها را محاسبه کنید.

فعالیت ۱۱: یک برنامه بنویسید که سیگنال سینوسی با طول ۵۰، فرکانس ۰.۰۸، دامنه ۲.۵ و فاز ۹۰ درجه ایجاد کرده و آن را ترسیم کنید. دوره تناوب این سیگنال چند است؟

فعالیت ۱۲: فرمان `stem` را یک نرتبه با فرمان `plot` و دفعه دیگر با فرمان `stairs` جایگزین کنید. شکل نمایش داده شده را در هر یک از این حالت‌ها با هم مقایسه کنید.

درس: برای ایجاد سیگنال تصادفی با طول N و توزیع یکنواخت در بازه $(0,1)$ از دستور $x = \text{rand}(1, N)$ در متلب استفاده می‌شود.

همچنین، دنباله تصادفی با طول N و با توزیع نرمال (دارای میانگین صفر و واریانس یک) با استفاده از دستور $x = randn(1, N)$ قابل ایجاد است.

فعالیت ۱۳: برنامه ای بنویسید که هم یک سیگنال تصادفی با طول ۱۰۰ و با توزیع یکنواخت در بازه $[-2, 2]$ و هم یک سیگنال تصادفی به طول ۷۵ با توزیع نرمال و میانگین صفر و واریانس ۳ ایجاد کند و هر دو را در قالب یک پنجره نمایش دهد.

$$R = \mu + (rand(1,100)) \times \sigma$$

فعالیت ۱۴: تابعی بنویسید که میانگین و واریانس دیتای داده شده به آن را محاسبه کند. سپس دو بردار تولید شده در فعالیت قبل را به آن بدهید و نتیجه را گزارش کنید.

فعالیت ۱۵: برنامه ای بنویسید که یک سیگنال تصادفی سینوسی با طول ۵۰ را ایجاد کرده که فاز و دامنه ی آن به ترتیب بین بازه ی ۰ تا 2π و ۰ تا ۸۰ باشد که هر دو مستقل از هم و دارای توزیع یکنواخت باشند. سپس به دلخواه ۵ نمونه از این سیگنال را انتخاب کرده و نمایش دهید.

$$x[n] = A * \cos(w_0 * n + \varphi)$$

فعالیت ۱۶: برنامه ای بنویسید که سیگنال نمایی به طول ۵۱ و بصورت عبارت مقابل دریافت کرده:

$$Sig = 2n(0.9^n)$$

سپس با نویزی که دارای میانگین صفر و انحراف معیار ۰.۸ و توزیع یکنواخت است جمع کرده، در نهایت این سیگنال بعنوان ورودی به سیستم زیر وارد شود، خروجی را محاسبه کرده و نمایش دهید.

$$y[n] = \frac{1}{3} (x(n-1) + x(n) + x(n+1))$$

آیا سیستم علی است؟ توضیح دهید.

این سیستم چه کاری انجام میدهد؟

سیگنال تصادفی سینوسی فعالیت قبل را نیز به عنوان ورودی به سیستم فوق وارد کرده و نتیجه را نمایش دهید.

آیا میتوانید با تغییر طول فیلتر (سیستم دیجیتال) نتیجه بهتری بدست آورید؟

فعالیت ۱۷: فرکانس لحظه‌ای در هر سیگنال سینوسی برابر است با مشتق فاز آن نسبت به زمان: برای ایجاد سیگنال‌هایی که فرکانس آن‌ها با زمان به صورت خطی تغییر میکند، نیاز است تا آرگومان آن‌ها تابعی از درجه‌ی دو نسبت به زمان باشد.

$$P = a \times n^2 + b \times n \Rightarrow f = \frac{dP}{dn} = 2an + b$$

مثالی از متغیر بودن فرکانس:

```
n = 0:100;  
a = pi/200; b = 0;  
arg = a * n.* n + b * n;  
x = cos(arg);  
stem(n,x)  
axis([0,100,-1.5,1.5]);  
title('Signal')  
xlabel('Time')  
ylabel('Amplitude')  
grid; axis;
```

فعالیت ۱۸: برنامه را طوری تغییر دهید که سیگنال سینوسی با فرکانس متغیر با مینیمم ۰.۲ و ماکزیمم ۰.۶ ایجاد کند.

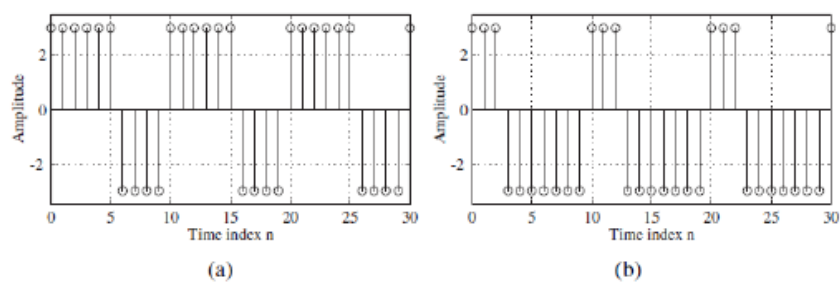
درس: با ترکیب سیگنال‌های پایه و استفاده از عملگرهای ساده، میتوان، سیگنال‌های پیچیده‌تری بوجود آورد. بعنوان مثال، AM را توسط سیگنال سینوسی با فرکانس بالا $x_H[n] = \cos(w_H n)$ برای سیگنال فرکانس پایین $x_L[n] = \cos(w_L n)$ انجام دهیم. سیگنال مدوله شده‌ی نهایی به صورت زیر است:

$$y[n] = A(1 + m \cdot x_L[n])x_H[n] = A(1 + m \cdot \cos(w_L n))\cos(w_H n)$$

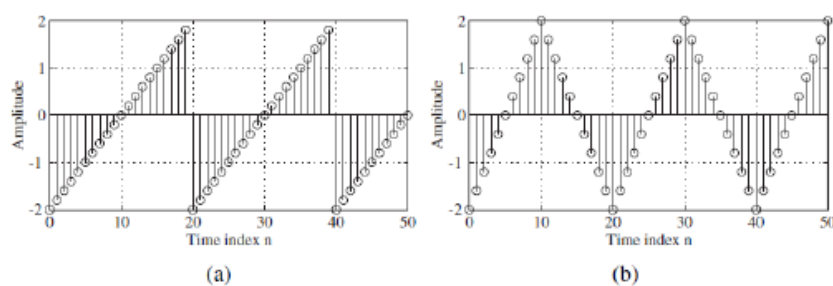
که در آن m اندیس مدولاسیون است و باید به گونه‌ای باشد که عبارت $(1 + m \cdot x_L[n])$ به ازای همه n ها، مثبت باشد.

فعالیت ۱۹: برنامه ای بنویسید که به ازای مقادیر مختلف اندیس مدولاسیون، فرکانس حامل و فرکانس سیگنال اصلی شکل موج را ترسیم کند، سپس نتیجه را تحلیل کنید.

فعالیت ۲۰: با استفاده از توابع square و sawtooth، برنامه‌ای بنویسید که خروجی آن، نمودارهای زیر باشد.



شکل ۱. شکل موج مربعی



آزمایش ۲: تحلیل سیگنال‌های زمان گسسته در حوزه زمان

در این آزمایش، بررسی خواص سیستم‌ها از لحاظ خطی بودن، پایداری، تغییرناپذیری با زمان، علی بودن و ... انجام خواهد شد.

در این قسمت با دستورات `filter`، `impz` و `conv` آشنا میشویم.

دستور `impz`، پاسخ ضربه را محاسبه میکند: ورودی یک ضربه است.

دستور `conv`، کانولوشن دو سیگنال را محاسبه میکند: همچنین دستور `deconv` عکس کانولوشن را برگرداند (سیگنال اصلی).

$$H(z) = \frac{b_1 z^{-1} + b_2 z^{-2} + b_3 z^{-3} + \dots}{1 + a_1 z^{-1} + a_2 z^{-2} + a_3 z^{-3} + \dots}$$

$$H(z) = \frac{2z^{-1} + 3z^{-2} + 5}{1 + 4z^{-1} - 6z^{-2} + z^{-3}} \Rightarrow \text{filter}([5, 2, 3], [1, 4, -6, 1], x)$$

بخش ۱: بررسی فیلتر میانگین یا هموار ساز

فعالیت ۱: تابع فیلتر میانگین در زیر نشان داده شده است.

$$y[n] = \frac{1}{M} \sum_{k=0}^{M-1} x[n-k]$$

ابتدا یک سیگنال سینوسی گسسته که یک سیگنال سینوسی با فرکانس بالا بعنوان نویز روی آن سوار شده است، را ایجاد کرده، سپس این سیگنال را به عنوان ورودی به سیستم فیلتر میانگین با رابطه فوق اعمال کرده و ورودی و خروجی را به ازای طول پنجره فیلتر (M) مختلف ترسیم کنید.

فعالیت ۲: فعالیت ۱ را برای $M = 2$ و فیلتر زیر تغییر دهید.

$$y[n] = 0.5(x[n] - x[n-1])$$

چه تاثیری بر سیگنال ورودی خواهد داشت؟

سیگنال ورودی را با یک سیگنال سینوسی که فرکانس افزایش و متغیر دارد جایگزین کنید: طول سیگنال 0-100 و فرکانس 0-0.5، سپس قسمت فعالیت ۱ و فعالیت ۲ را دوباره با این ورودی جدید اجرا کنید.

بخش ۲: بررسی یک سیستم غیرخطی ساده

فعالیت ۳: سیگنال غیرخطی زیر را در نظر بگیرید:

با استفاده از اعمال ورودی‌های سینوسی با فرکانس‌های مختلف، خروجی را مشاهده نمایید. سیگنال خروجی چگونه با تغییر فرکانس تغییر میکند؟ اط لحاظ تئوری نیز بررسی کنید.

$$y[n] = x[n]^2 - x[n-1]x[n+1]$$

یک مقدار DC به سیستم فوق اعمال کرده و خروجی را بررسی کنید.

یک سینوسی به سیستم فوق اعمال کرده و خروجی را بررسی کنید.

یک سینوسی به همراه یک مقدار DC به سیستم فوق اعمال کرده و خروجی را بررسی کنید.

نتیجه سه بخش فوق را در جهت بررسی خطی یا غیر خطی بودن سیستم بررسی کنید. (آیا جمع پاسخ دو حالت اول و دوم با حاصل قسمت سوم یکی است؟)

فعالیت ۴: سیستم زیر را از لحاظ خطی بودن بررسی کنید. (اعمال ترکیب خطی از دو ورودی به سیستم و مقایسه با ترکیب خطی خروجی‌ها در صورت اعمال ورودی‌ها بطور مستقل و مقایسه نتیجه)

$$y[n] - 0.4y[n-1] - 0.75y[n-2] = 2.24x[n] + 2.49x[n-1] + 2.24x[n-2]$$

فعالیت ۵: سیستم زیر را نیز از لحاظ خطی بودن بررسی کنید.

$$y[n] = x[n] \times x[n-1]$$

فعالیت ۶: در دو فعالیت قبل، در حالت شرایط اولیه غیر صفر نیز نتیجه را بررسی کنید. چه نتیجه‌ای میگیرید؟

بخش ۳: بررسی خاصیت تغییرپذیری یا تغییرناپذیری با زمان

فعالیت ۷: سیستم مطرح شده زیر را از لحاظ TI بودن بررسی کنید.

$$y[n] - 0.4y[n - 1] - 0.75y[n - 2] = 2.24x[n] + 2.49x[n - 1] + 2.24x[n - 2]$$

برای این کار باید ورودی‌های $x[n]$ و $x[n - D]$ را به سیستم اعمال کرده و خروجی‌های متناظر $y_1[n]$ و $y_2[n]$ را بدست آورده و سپس بررسی کنید که آیا $y_2[n + D]$ با $y_1[n]$ برابر است یا خیر.

این کار را برای مقادیر مختلف از D تکرار کنید و نتیجه را نمایش دهید.

فعالیت ۸: فعالیت قبل را برای حالت، شرایط اولیه غیر صفر نیز انجام دهید. آیا در اینصورت سیستم تغییرناپذیر با زمان است؟

فعالیت ۹: سیستم قبل را با سیستم زیر جایگزین کرده و خاصیت تغییرناپذیری با زمان را بررسی کنید.

$$y[n] = nx[n] + x[n - 1]$$

بخش ۴: محاسبه پاسخ ضربه

فعالیت ۱۰: پاسخ ضربه سیستم زیر را با استفاده از دستور $impz(num, den, N)$ محاسبه کرده و ۵۰ سمپل اولیه-ی آن را نمایش دهید.

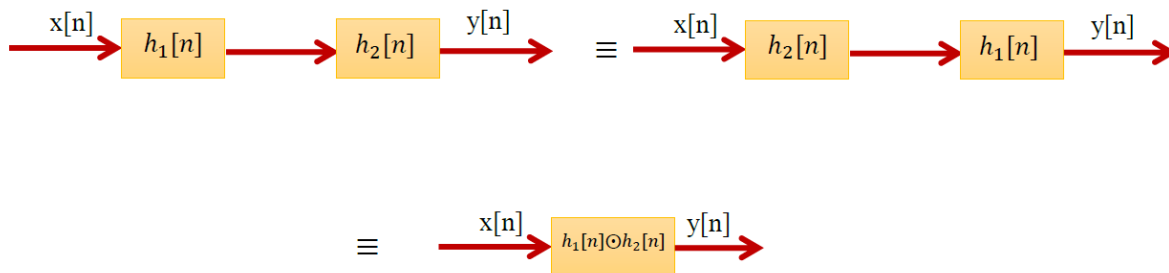
$$y[n] + 0.71y[n - 1] - 0.46y[n - 2] - 0.62y[n - 3] = 0.9x[n] - 0.45x[n - 1] + 0.35x[n - 2] + 0.2x[n - 3]$$

سپس بجای استفاده از دستور $impz$ ، از دستور $filter$ پاسخ ضربه‌ی سیستم را محاسبه کنید و ۴۰ سمپل اول را نمایش دهید. شکل خروجی را با قسمت قبل مقایسه کنید.

فعالیت ۱۱: برنامه‌ای بنویسید که پاسخ پله‌ی سیستم فوق را محاسبه و ۴۰ سمپل از آن را نمایش دهد.

بخش ۵: سری کردن سیستم‌ها

در عمل می‌توان سیستم‌های علی و LTI مرتبه بالا را میتوان با استفاده از سری کردن سیستم‌های مرتبه پایین-تر پیاده‌سازی کرد. این کار چه مزیتی دارد؟ در شکل زیر اتصال سری را مشاهده میکنید.



فعالیت ۱۲: سیستم مرتبه ۴ زیر را پیاده سازی کرده

$$y[n] + 1.6y[n-1] + 2.28y[n-2] - 1.35y[n-3] + 0.68y[n-4] = 0.06x[n] - 0.19x[n-1] + 0.27x[n-2] - 0.26x[n-3] + 0.12x[n-4]$$

سپس نشان دهید که آیا میتوان این سیستم را با سری کردن دو سیستم زیر پیاده سازی کرد، یا خیر؟

$$y_1[n] + 0.9y_1[n-1] + 0.8y_1[n-2] = 0.3x[n] - 0.2x[n-1] + 0.4x[n-2]$$

$$y_2[n] + 0.7y_2[n-1] + 0.85y_2[n-2] = 0.2y_1[n] - 0.5y_1[n-1] + 0.3y_1[n-2]$$

فعالیت ۱۳: این برنامه را برای ورودی سینوسی یک مرتبه با شرایط اولیه صفر و یک مرتبه با شرایط اولیه غیر صفر اجرا کنید و نتیجه خود را بیان کنید.

بخش ۶: کانولوشن

میدانیم که یکی از خواص سیستم‌های LTI این است که پاسخ سیستم را میتوان با استفاده از کانولوشن ورودی محدود با پاسخ ضربه سیستم بدست آورد. در این آزمایش این خاصیت را بررسی میکنیم.

فعالیت ۱۴: برنامه‌ای بنویسید که پاسخ سیستم قبل را به ورودی‌های مختلف ضربه، پله، سینوسی، یک مرتبه با استفاده از کانولوشن و یک مرتبه با استفاده از دستور *filter* بدست آورده و خروجی را در دو حالت مقایسه کند. آیا خروجی در دو حالت یکسان هستند؟ دلیل اضافه کردن صفر به ورودی قبل از فیلتر کردن چیست؟

بخش ۷: بررسی پایداری سیستم‌ها

میدانیم که یک سیستم LTI زمانی نسبت به ورودی محدود پایدار است که پاسخ ضربه‌ی آن محدود باشد. بنابراین شرط لازم برای پایداری سیستم LTI و IIR این است که با افزایش اندیس زمان، پاسخ ضربه به سمت صفر همگرا شود.

شرط پایداری زیر را بررسی و اثبات کنید:

$$\sum_{n=-\infty}^{\infty} |h[n]| < \infty$$

فعالیت ۱۵: برای سیستم زیر خاصیت پایداری را بررسی کنید:

$$y[n] + 1.5y[n-1] + 0.9y[n-2] = x[n] - 0.8x[n-1]$$

فعالیت ۱۶: برای سیستم زیر خاصیت پایداری را بررسی کنید:

$$y[n] = x[n] - 4x[n-1] + 3x[n-2] + 1.76y[n-1] - y[n-2]$$

آزمایش ۳: سیگنال‌های زمان گسسته در حوزه فرکانس

در این آزمایش به بررسی خواص سیگنال‌های زمان گسسته از منظر فرکانس می‌پردازیم. به همین منظور، سه نوع نمایش معروف و پرکاربرد تبدیل فوریه گسسته (DTFT)، تبدیل فوریه سریع (FFT) و تبدیل Z استفاده خواهد شد و ویژگی‌های آن‌ها بررسی خواهند شد.

برخی توابع متلی مورد استفاده عبارتند از:

Elementary matrices and matrix manipulation:

ماتریس‌های اولیه و به کار بردن ماتریس:

fliplr	i	pi	zeros	:
--------	---	----	-------	---

Elementary functions:

توابع اولیه:

abs	angle	conj	exp	imag	real
rem					

Polynomial and interpolation functions:

توابع چندجمله‌ای و تعاملی :

conv

Data analysis and fourier transform functions:

آنالیز داده و توابع تبدیل فوریه:

fft	ifft	max	min
-----	------	-----	-----

Signal processing toolbox:

جعبه ابزار پردازش سیگنال:

freqz	impz	residuez	Tf2zp	Zp2sos	Zp2tf
zplane					

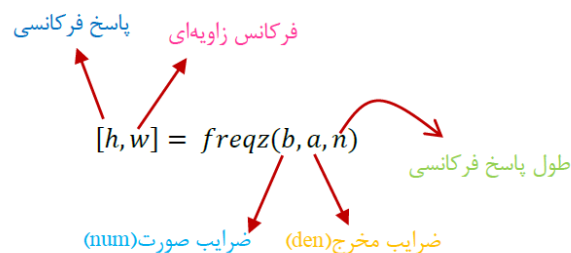
.^ , ^

$\text{fliplr} \Rightarrow \text{flip rotate (left to right)}$

$\text{rem} \Rightarrow$ باقیمانده

fft , ifft

$\text{freqz} \Rightarrow$ محاسبه‌ی پاسخ فرکانسی تابع تبدیل



$$[z, p, k] = tf2zp \quad , \quad zp2tf$$

$$H(s) = \frac{B(s)}{A(s)} = \frac{b_1 s^{n-1} + \dots + b_{n-1} s + b_n}{a_1 s^{m-1} + \dots + a_{m-1} s + a_m} \quad \rightarrow \quad H(s) = \frac{Z(s)}{P(s)} = k \frac{(s - z_1)(s - z_2) \dots (s - z_m)}{(s - p_1)(s - p_2) \dots (s - p_n)}$$

zp2sos

$$H(z) = k \frac{(z - z_1)(z - z_2) \dots (z - z_n)}{(z - p_1)(z - p_2) \dots (z - p_m)} \quad \rightarrow \quad H(z) = g \prod_{k=1}^L H_k(z) = g \prod_{k=1}^L \frac{b_{0k} + b_{1k} z^{-1} + b_{2k} z^{-2}}{1 + a_{1k} z^{-1} + a_{2k} z^{-2}}$$

$$\text{sos} = \begin{bmatrix} b_{01} & b_{11} & b_{21} & 1 & a_{11} & a_{21} \\ b_{02} & b_{12} & b_{22} & 1 & a_{12} & a_{22} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ b_{0L} & b_{1L} & b_{2L} & 1 & a_{1L} & a_{2L} \end{bmatrix}$$

بخش ۱: محاسبه‌ی تبدیل فوریه گسسته DTFT

میدانیم که تبدیل فوریه سیگنال گسسته، تابعی پیوسته در حوزه فرکانس است. روابط مربوط به تبدیل فوریه گسسته و معکوس آن عبارتند از:

$$X(\omega) = \sum_{n=-\infty}^{n=\infty} x[n] e^{-j\omega n}$$

$$x[n] = \frac{1}{2\pi} \int_{\omega=-\pi}^{\omega=\pi} X(\omega) e^{j\omega n} d\omega$$

به دو دلیل امکان پیاده‌سازی تبدیل فوریه گسسته در متلب وجود ندارد. دلیل اول این است که در رابطه تبدیل فوریه طول سیگنال نامحدود است لذا در کامپیوتر قابل نمایش و محاسبه نیست. دلیل دوم اینکه تبدیل فوریه‌ی گسسته تابعی پیوسته از ω است، اما در متلب مقادیر بصورت گسسته ذخیره شده و نمایش داده میشوند. بنابراین برای محاسبه تبدیل فوریه‌ی گسسته میتوانیم از دستور freqz استفاده نمود و در ضرایب فرکانسی گسسته، مقادیر تبدیل فوریه را مشاهده کنیم.

فعالیت ۱: پاسخ فرکانسی تابع تبدیلی با ضرایب زیر را محاسبه کنید.

$$num = [2 \ 1] \quad den = [1 \ -0.6]$$

دامنه و فاز تبدیل فوریه را جداگانه محاسبه کنید. آیا تبدیل فوریه متناوب است؟ اگر بله دوره تناوب آن چیست؟ قسمت‌های نمایش داده شده از لحاظ تقارن چگونه هستند؟

فعالیت ۲: برنامه را طوری تغییر دهید که پاسخ فرکانسی سیگنال محدود زیر را نمایش دهد. همچنین فاز را بصورت درجه محاسبه کرده و نمایش دهید.

$$R[n] = [1 \ 4 \ 7 \ 10 \ 13 \ 16 \ 19 \ 22 \ 25 \ 28]$$

بخش ۲: شیفت زمانی تبدیل فوریه گسسته

$$x[n - n_0] \xrightarrow{F} e^{-jn_0\omega} X(e^{j\omega})$$

فعالیت ۳: پاسخ فرکانسی سیگنال شیفت یافته فعالیت ۲ را محاسبه کنید. شیفت زمانی چه تاثیری در تبدیل فوریه سیگنال داشته است؟

فعالیت ۴: به ازای دو رشته متفاوت و دو مقدار مختلف شیفت زمانی، خروجی‌ها را بررسی کنید و نتیجه را بیان کنید.

بخش ۳: شیفت فرکانسی تبدیل فوریه گسسته

$$e^{j\omega_0 n} x[n] \xrightarrow{F} X(e^{j(\omega - \omega_0)})$$

فعالیت ۵: به ازای مقادیر مختلف شیفت فرکانسی، این ویژگی را بررسی کنید. (حداقل دو مقدار)

تاثیر شیفت فرکانسی در تبدیل فوریه چیست؟

بخش ۴: ویژگی کانولوشن و ضرب تبدیل فوریه گسسته

عمل کانولوشن یکی از کاربردی ترین ابزارها در تحلیل سیستم های LTI است. بنابراین دانستن رفتار حوزه فرکانس دو سیگنالی که با هم کانوالو میشوند مفید خواهد بود.

فعالیت ۶: ویژگی معادل بودن کانولوشن در حوزه زمان با ضل در حوزه فرکانس را با استفاده از دو سیگنال مختلف و دلخواه بررسی کنید.

فعالیت ۷: ویژگی (مدولاسیون) یا معادل بودن ضرب در حوزه زمان با کانولوشن در حوزه فرکانس را با استفاده از دو سیگنال مختلف دلخواه بررسی و تحقیق نمایید.

بخش ۵: ویژگی معکوس کردن زمان

میخواهیم بدانیم که اگر متغیر زمان در سیگنال معکوس شود، تبدیل فوریه آن چه تغییری میکند؟

$$x[-n] \xrightarrow{F} X(e^{-j\omega})$$

فعالیت ۸: با استفاده از دو سیگنال دلخواه خاصیت فوق را بررسی و تحقیق نمایید.

بخش ۶: تبدیل فوریه سریع FFT

در آزمایش قبل دیدیم که تبدیل فوریه گسسته با روابط مذکور قابل محاسبه یست و باید در مقادیر گسسته ای از فرکانس، تبدیل فوریه را بیابیم. برای این منظور می توانیم تبدیل فوریه گسسته را در مقادیر گسسته ای فرکانس با فاصله مساوی در بازه $0 \leq \omega \leq 2\pi$ به ازای $\omega_k = \frac{2\pi k}{N}, k = 0, 1, \dots, N-1$ محاسبه کنیم، روابط بصورت زیر خواهد بود:

$$X[k] = \sum_{n=0}^{M-1} x[n] e^{-j2\pi kn/N}; k = 0, 1, \dots, N-1$$

$$x[n] = \sum_{k=0}^{N-1} x[n] e^{j2\pi kn/N}; n = 0, 1, \dots, M-1$$

بطور خلاصه:

$$\text{DFS} \Rightarrow x[n] \rightarrow a_k = a_{k+N}$$

$$\text{DTFT} \Rightarrow x[n] \rightarrow X(e^{j\omega}) \quad \text{پیوسته، طول نامحدود}$$

$$\text{DFT} \Rightarrow x[n] = X[k] \quad \text{گسسته، طول محدود}$$

$$\text{FFT} \Rightarrow \text{DFT "N"points, } N = 2^Y \text{ \& } \geq \text{طول سیگنال}$$

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\left(\frac{2\pi}{N}\right)kn}, \quad k = 0, 1, \dots, N-1$$

برای پیاده سازی در متلب از دستور $\text{fft}(x, L)$ استفاده می شود، که DFT، L نقطه ای سیگنال x را محاسبه می کند. دستور ifft هم معکوس این تبدیل را ارائه می دهد.

برای رشته محدود $x[n]$ ، تبدیل فوریه $X[k]$ با دستور $\text{fft}(x)$ با طولی برابر $x[n]$ ارائه میشود.

فعالیت ۹: برنامه ای بنویسید که تبدیل فوریه L نقطه ای سیگنال ورودی دلخواه $x[n]$ دارای N نقطه را ($L \gg N$) محاسبه و رسم کند، سپس تبدیل فوریه معکوس L نقطه ای را نیز رسم کند. به ازای چند ورودی دلخواه و مقادیر مختلفی از L و N خروجی ها را مشاهده کنید. چه نتیجه ای می گیرید؟

بخش ۷: ویژگی های شیف و کانولوشن چرخشی fft

فعالیت ۱۰: تابعی برای انجام شیف چرخشی بنویسید. نام تابع را circshift1 بگذارید. در یک فایل دیگر از این تابع بر روی یک سیگنال دلخواه استفاده کنید. اگر مقدار شیف از طول سیگنال بیشتر باشد چه اتفاقی می افتد؟ خاصیت شیف چرخشی تبدیل فوریه گسسته چیست؟

فعالیت ۱۱: تابعی برای انجام کانولوشن چرخشی بنویسید. نام تابع را *circconv1* بگذارید. در یک برنامه دیگر از این تابع برای انجام کانولوشن چرخشی بر روی دو سیگنال دلخواه استفاده کنید. خاصیت کانولوشن چرخشی تبدیل فوری را توضیح دهید.

فعالیت ۱۲: با استفاده از کانولوشن چرخشی، کانولوشن خطی را برای دو سیگنال با طول متفاوت پیاده‌سازی کنید. چه خاصیتی بین کانولوشن چرخشی و کانولوشن خطی برقرار است؟

فعالیت ۱۳: برنامه‌ای بنویسید که کانولوشن دو سیگنال را با استفاده از تبدیل فوری تک‌تک آن‌ها محاسبه کند.

آزمایش ۴: تحلیل سیگنال‌های زمان گسسته در حوزه فرکانس

می‌دانیم که رفتار هر سیستم LTI، با در دست داشتن پاسخ ضربه‌ی آن به طور کامل مشخص می‌شود. همچنین در آزمایش قبل، تبدیل فوریه‌ی پاسخ ضربه (پاسخ فرکانسی) را بررسی کردیم. در این آزمایش، ابتدا با تبدیل z پاسخ ضربه، که تابع تبدیل نام دارد آشنا می‌شویم. سپس ویژگی‌ها و انواع سیستم‌ها را از نظر پاسخ فرکانسی بررسی می‌کنیم. همچنین با محاسبه پاسخ فرکانسی و پاسخ ضربه از روی تابع تبدیل، پایداری سیستم، ویژگی‌های مختلف سیستم علی و LTI پایدار، بررسی نمودار صفرها و قطب‌های تابع تبدیل و بررسی پایداری با استفاده از این نمودار، آشنا خواهیم شد.

برخی دستورات متلب مورد نیاز:

Signal processing toolbox:

جعبه ابزار پردازش سیگنال:

freqz	impz	residuez	Tf2zp	Zp2sos	Zp2tf
zplane					

Elementary functions:

توابع اولیه:

abs	angle	imag	Log10	real	Real
-----	-------	------	-------	------	------

بخش ۱: رسم قطب‌ها و صفرهای تبدیل Z سیستم

میدانیم که تبدیل فوریه گسسته، همان تبدیل z روی دایره واحد است. یعنی در حالت کلی z عددی مختلط خواهد بود و در ازای $z = e^{j\omega}$ تبدیل Z به تبدیل فوریه گسسته تبدیل می‌شود. بنابراین، واضح است که دستور freqz تبدیل z را محاسبه میکند اما تنها بر روی دایره واحد.

فعالیت ۱: دستور zplane را در دو حالت مختلف شرح دهید. (ورودی تابع تبدیل باشد و یا ورودی بردارهای صفر و قطب باشد). شرط پایداری را نیز بررسی کنید.

فعالیت ۲: برنامه‌ای بنویسید که نمودار صفرها و قطب‌های تابع تبدیل زیر را نشان دهد.

$$G(z) = \frac{2 + 5z^{-1} + 9z^{-2} + 5z^{-3} + 3z^{-4}}{5 + 45z^{-1} + 2z^{-2} + z^{-3} + z^{-4}}$$

فعالیت ۳: با استفاده از دستور tf2zp، مکان قطب‌ها و صفرهای این سیستم را مشخص کنید.

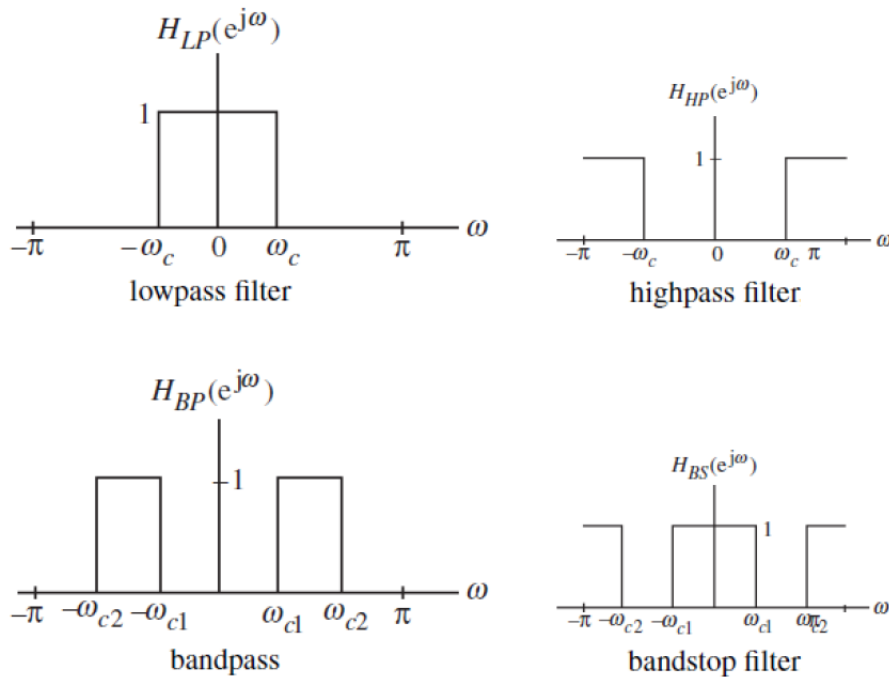
فعالیت ۴: با اعمال دستورات زیر نتیجه را بررسی کنید و تحلیل و مقایسه کنید.

$$[Z, P, K] = tf2zp(b, a); \quad [b, a] = zp2tf(z, p, k); \quad [r, p, k] = residuez(b, a);$$

$$\frac{B(z)}{A(z)} = \frac{r(1)}{1 - p(1)z^{-1}} + \dots + \frac{r(n)}{1 - p(n)z^{-1}} + k(1) + k(2)z^{-1} + \dots + k(m - n + 1)z^{-(m-n)}$$

بخش ۲: بررسی ویژگی سیستم‌ها، انواع فیلترها:

یکی از مهمترین کاربردهای پردازش سیگنال دیجیتال، فیلتر کردن سیگنال‌های مختلف به منظور حذف اجزای نامطلوب آنها است. پاسخ فرکانسی چهار فیلتر ایده‌آل را در شکل زیر مشاهده میکنید.



پیاده‌سازی فیلترهای ایده‌آل در عمل امکان‌پذیر نیست، زیرا باید طول پاسخ ضربه‌ی چنین فیلترهایی در حوزه زمان بینهایت باشد که غیر ممکن است. بنابراین از فیلترهای غیر ایده‌آل استفاده میشود که پاسخ ضربه‌ی با طول محدودی دارند، همچنین برای پیاده‌سازی باید فیلتر علی باشد، تخمینی از پاسخ ضربه‌ی محدود برای فیلتر پایین‌گذر بصورت زیر بدست می‌آید:

$$h_{LP}[n] = \frac{\sin(\omega_c(n - N/2))}{\pi(n - N/2)}$$

فعالیت ۵: پاسخ ضربه‌ی تخمین زده شده فیلتر FIR پایین‌گذر را برای فرکانس قطع ۰.۲۵ و به تعداد دلخواه نمونه، پیاده‌ازی و رسم نمایید. برای طول ۲۰ و فرکانس قطع ۰.۴۵ نیز پیاده‌سازی نمایید.

بخش ۳: محاسبه بهره تابع تبدیل

معمولا بهره تابع تبدیل بر حسب دسی‌بل محاسبه و رسم میشود و فرکانس قطع 3-dB آن لحاظ میشود. منظور محاسبه‌ی بهره تابع تبدیل بر حسب دسی‌بل، از روش moving average استفاده میشود.

فعالیت ۶: برنامه‌ای بنویسید که gain تابع تبدیل فیلتر بالاگذر زیر را برای $M=5$ بر حسب دسی‌بل محاسبه و رسم کند. فرکانس 3dB را مشخص کنید.

$$H(z) = \frac{1}{M} \sum_{n=0}^{M-1} (-1)^n z^{-n}$$

بخش ۴: بررسی پایداری فیلترها

پایداری فیلترهای IIR یکی از پارامترهای مهم در طراحی این نوع فیلترها می‌باشد. میدانیم زمانی فیلتر پایدار خواهد بود که تمامی قطب‌های تابع تبدیل آن درون دایره واحد قرار داشته باشند. اگرچه می‌توان با استفاده از دستورات zplane، قطب‌ها را مشاهده و پایداری را تحقیق کرد، اما زمانی که قطب‌ها خیلی نزدیک به دایره واحد باشند، این برای بررسی پایداری، مناسب و دقیق نخواهد بود. با استفاده از دستور poly2rc میتوانیم پایداری را بصورت دقیق‌تری بررسی کنیم.

فعالیت ۷: دو سیستم با تابع تبدیل زیر را در نظر بگیرید:

$$H_1(z) = \frac{1}{1 - 1.848z^{-1} + 0.85z^{-2}}$$

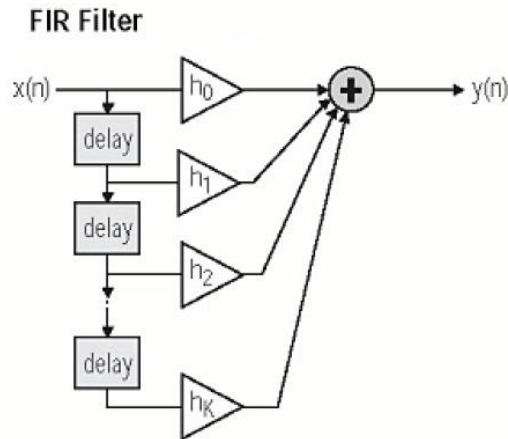
$$H_2(z) = \frac{1}{1 - 1.85z^{-1} + 0.85z^{-2}}$$

با دستور zplane محل صفر و قطب را نشان دهید. آیا امکان تشخیص پایداری وجود دارد؟

سپس با دستور poly2rc پایداری را بررسی کنید.

آزمایش ۵: طراحی فیلترهای دیجیتالی FIR

در طراحی فیلتر، پس از انتخاب نوع فیلتر، باید با توجه به مشخصات مطلوب، درجه تابع تبدیل را تعیین کرده و سپس تابع تبدیل را با توجه به مشخصات فیلتر تعیین کنیم.



دستورات متلب مورد نیاز:

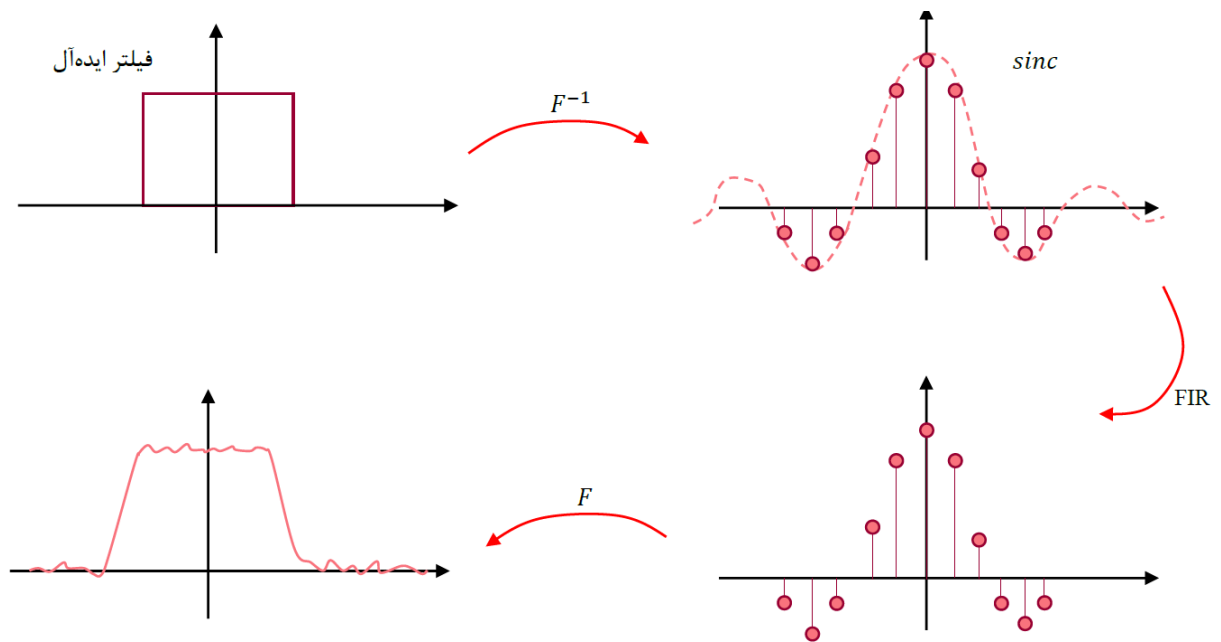
Signal processing toolbox:

جعبه ابزار پردازش سیگنال:

blackman	butter	buttord	chebwin	cheb1ord	Cheb2ord	Cheb1
Cheb2	ellip	ellipord	fir1	fir2	Firpm	firpmord
freqz	hanning	hamming	kaiser			

بخش ۱: طراحی فیلتر FIR

ساده‌ترین روش برای طراحی فیلتر FIR، این است که از پاسخ فرکانسی فیلتر مطلوب، تبدیل فوری معکوس گرفته و سپس تعداد محدودی از نقاط آن را انتخاب کنیم. اما این کار باعث ایجاد ریب‌هایی در پاسخ فرکانسی می‌شود که اثر Gibbs نام دارد. برای حذف این و یا کاهش این اثر، پاسخ ضربه نامحدود را از پنجره‌ی محدود و مناسب عبور می‌دهیم. در متلب فرامینی وجود دارد که با استفاده از آن‌ها می‌توان بردار ضرایب پاسخ ضربه فیلتر FIR با مشخصات مطلوب را بدست آورد.



برای طراحی فیلتر FIR ابتدا باید درجه فیلتر FIR مشخص شود. برای این منظور از دستورات زیر استفاده می‌شود:

$$[N, \omega_n, \text{beta}, \text{ftype}] = \text{kaiserord}(\text{Fedge}, \text{Aval}, \text{dev})$$

or

$$[N, \omega_n, \text{beta}, \text{ftype}] = \text{kaiserord}(\text{Fedge}, \text{Aval}, \text{dev}, \text{Fs})$$

Fedge = فرکانس‌های مرزی , Aval = مقدار دامنه در هر باند , dev = مقدار ریبیل در هر باند , Fs = فرکانس نمونه‌برداری

beta = فاکتور شکل‌دهی , ftype = نوع فیلتر



میزان تضعیف لوب‌های فرعی

بعد از تعیین مرتبه‌ی فیلتر FIR، با استفاده از دستورات زیر، ضرایب پاسخ ضربه‌ی فیلتر را بدست می‌آوریم:

$$b = \text{fir1}(N, \omega_n, \text{'filtertype'})$$

$$b = \text{fir1}(N, \omega_n, \text{taper})$$

$$\text{taper} = \text{blackman}(N) , \text{taper} = \text{hamming}(N) , \text{taper} = \text{hanning}(N) , \text{taper} = \text{chebwin}(N) , \text{taper} = \text{kaiser}(N, \text{beta})$$

$$\text{taper} = \text{triang}(N) , \text{taper} = \text{rectwin}(N)$$

$$b = \text{fir1}(N, W_n)$$

$$b = \text{fir1}(N, W_n, \text{'high'})$$

$$b = \text{fir1}(N, W_n, \text{'stop'})$$

$$b = \text{fir1}(N, W_n, \text{taper})$$

نکته: همان‌گونه که پیش‌تر توضیح داده شد، برای حذف اثر نوسان در فیلتر FIR از پنجره استفاده می‌کنیم. در حالت استفاده بدون taper، فرمان به صورت پیش‌فرض از پنجره‌ی همپینگ hamming استفاده خواهد کرد. اما در صورت نیاز به استفاده از پنجره‌های دیگر، می‌توان با فرمان‌های زیر، پنجره‌های دیگری نیز ایجاد کرد.

$$taper = blackman(N) \quad taper = hamming(N) \quad taper = hanning(N)$$

$$taper = chebwin(N) \quad taper = kaiser(N, beta)$$

فعالیت ۱: با استفاده از دستور fir1، فیلتر FIR پایین‌گذر با فاز خطی و دارای مشخصات مطرح شده را طراحی کنید: فرکانس مرز باند عبور 2kHz، فرکانس مرز باند توقف 2.5kHz، فرکانس نمونه برداری 10kHz، ریپل باند عبور و باند توقف $\delta_s = \delta_p = 0.005$.

پس از طراحی فیلتر، گین پاسخ فرکانسی و فاز آن را رسم کنید. آیا فیلتر طراحی شده دارای مشخصات مطلوبی می‌باشد؟ با تغییر چه پارامتری می‌توان فیلتر را مطلوب‌تر طراحی کرد؟

فعالیت ۲: فعالیت ۱ را بار دیگر با استفاده از پنجره‌های hamming و blackman انجام دهید و نتایج را مقایسه کنید و شرح دهید.

فعالیت ۳: الف) یک سیگنال سینوسی ساده با فرکانس $f_0 = 50$ هرتز و طول یک ثانیه را ایجاد کنید.

ب) DFT سیگنال را با استفاده از پنجره‌ی مستطیلی محاسبه کنید.

ج) طیف حاصل را نمایش دهید.

فعالیت ۴: الف) سیگنال مشابه فعالیت قبل را تولید کنید.

ب) DFT سیگنال را با استفاده از پنجره هانینگ محاسبه و طیف آن را مشاهده کنید.

ج) طیف پنجره مستطیلی و هانینگ را در یک شکل مقایسه کنید.

فعالیت ۵: الف) پنجره کیزر با پارامترهای مختلف $\beta = 2, \beta = 10$ را اعمال کنید.

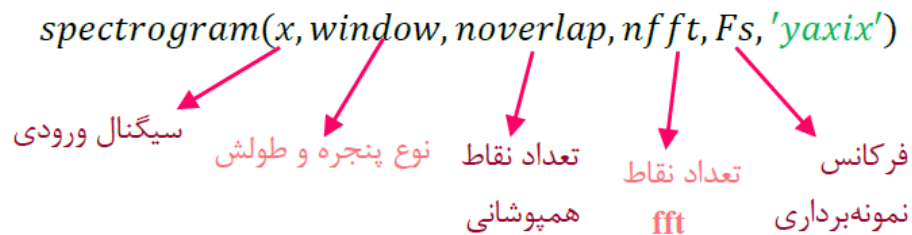
ب) طیف سیگنال را برای هر دو مقدار β محاسبه کرده و نمایش دهید.

ج) اثر تغییرات پارامتر β را روی پهنای باند طیف بررسی کنید.

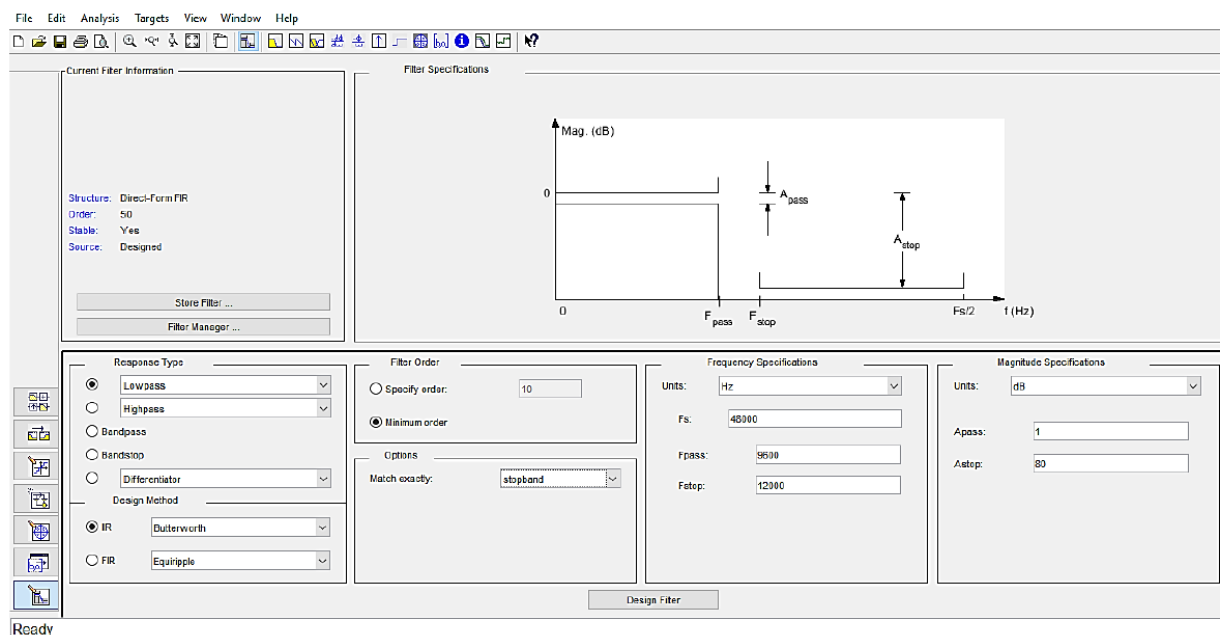
فعالیت ۶: یک سیگنال سینوسی با فرکانس غیر مضربی از نرخ نمونه‌برداری (مانند ۴۵.۵ هرتز) ایجاد کنید. سپس از پنجره‌های مختلف (مستطیلی، هانینگ، همینگ، کیزر) استفاده کنید و طیف آن‌ها را نمایش دهید. در هر پنجره نشت فرکانسی را بررسی و مقایسه کنید.

فعالیت ۷: یک فعالیت در مورد تاخیر گروه و انواع پنجره‌گذاری بر روی سیگنال صوتی طراحی نمایید.

فعالیت ۸: در مورد تبدیل فوریه زمان کوتاه (STFT) توضیح دهید. فعالیتی در مورد استفاده از دستور زیر طراحی نمایید.



بخش ۲: استفاده از تولباکس filter Designer با fdatool



فعالیت ۹: الف) در FDATool یک فیلتر پایین‌گذر با مشخصات زیر طراحی کنید:

- فرکانس نمونه‌برداری: ۱۰۰۰ هرتز
- فرکانس قطع: ۱۰۰ هرتز
- نوع فیلتر: FIR (با روش طراحی Window)

ب) در محیط متلب یک سیگنال تصادفی ایجاد کنید و فیلتر طراحی شده را اعمال کنید.

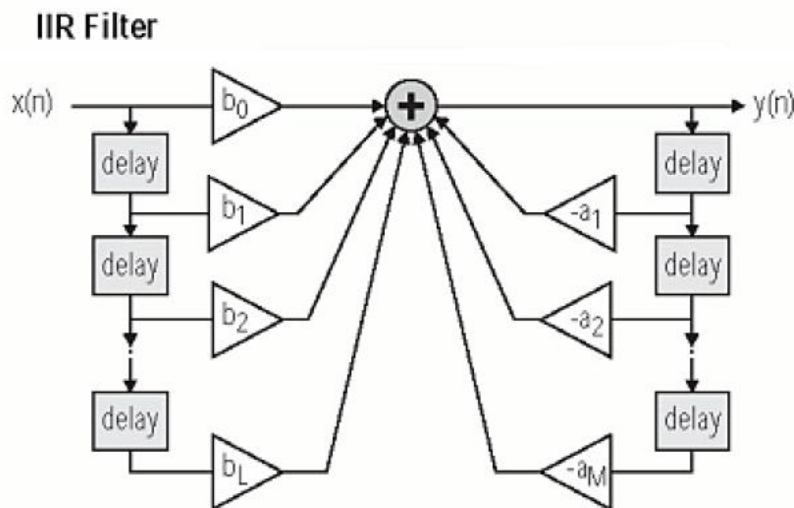
فعالیت ۱۰: الف) در FDATool یک فیلتر میانگذر با مشخصات زیر طراحی کنید:

- فرکانس نمونه‌برداری: ۵۰۰۰ هرتز
- فرکانس‌های قطع: ۱۰۰۰ هرتز و ۲۰۰۰ هرتز
- نوع فیلتر: FIR (با روش Equiripple)

ب) یک سیگنال شامل نویز و چند فرکانس مختلف ایجاد کنید و فیلتر میان‌گذر را برای حذف نویز و بخش‌های غیر مطلوب سیگنال استفاده کنید.

آزمایش ۶: طراحی فیلترهای دیجیتال IIR

طراحی فیلتر دیجیتال به معنی بدست آوردن تابع تبدیلی است که پاسخ فرکانسی آن، تقریب از پاسخ فرکانسی مطلوب باشد. پس از طراحی تابع تبدیل، آن را پیاده سازی می کنند. در این آزمایش با اصول طراحی فیلتر IIR و بدست آوردن توابع تبدیل مورد نظر با گین مشخص آشنا میشویم.



برخی دستورات متلب مورد نیاز:

Signal processing toolbox:

جعبه ابزار پردازش سیگنال:

blackman	butter	buttord	chebwin	cheb1ord	Cheb2ord	Cheb1
Cheb2	ellip	ellipord	fir1	fir2	firpm	firpmord
freqz	hanning	hamming	kaiser			

معروف ترین روش طراحی فیلتر IIR، بر مبنای روش bilinear از تابع تبدیل پیوسته نمونه است. تابع تبدیل آنالوگ معمولاً یکی از چهار نوع باترورث (Butterworth)، چبی شف (Chebyshev) نوع ۱، چبی شف نوع ۲ و elliptic است.

elliptic	Cheb2	Cheb1	Butterworth
(۱) دارای ریبیل مساوی در باند عبور و قطع (۲) ریبیل قابل تنظیم (۳) شیب زیاد در باند گذار	ریبیل در باند قطع	(۱) دارای شیب زیاد در باند گذار (۲) ریبیل در باند عبور	(۱) بدون ریبیل در باند قطع و باند عبور (۲) فاز غیر خطی

بخش ۱: تعیین درجه فیلتر IIR

اولین گام پس از انتخاب نوع فیلتر مورد نیاز، تعیین درجه‌ی تابع تبدیل آن بر اساس ویژگی‌های مورد نیاز است. فرامین زیر در متلب به منظور تعیین درجه‌ی چهار نوع فیلتر ذکر شده، قرار داده شده است.

فعالیت ۱: توضیح دهید که هر یک از پارامترهای فرامین بالا نشان‌گر چیست؟

$$[N, W_n] = \text{buttord}(W_P, W_S, R_P, R_S),$$

$$[N, W_n] = \text{cheb1ord}(W_P, W_S, R_P, R_S),$$

$$[N, W_n] = \text{cheb2ord}(W_P, W_S, R_P, R_S),$$

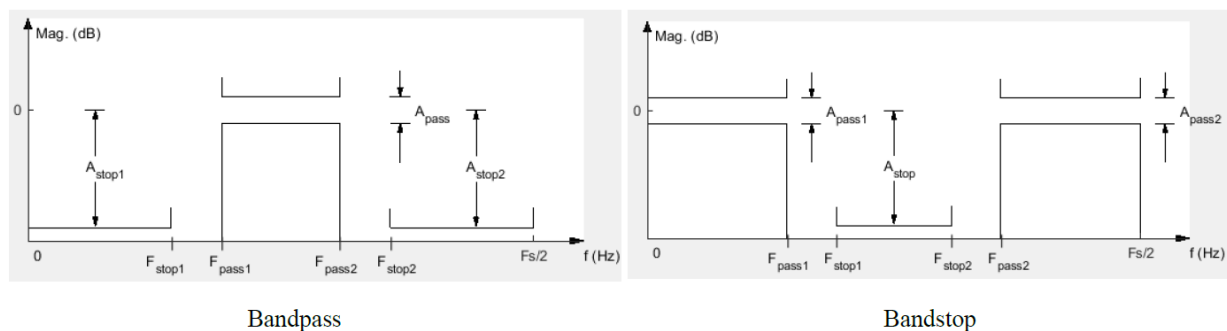
$$[N, W_n] = \text{ellipord}(W_P, W_S, R_P, R_S),$$

فعالیت ۲: برنامه‌ای بنویسید که کمترین درجه‌ی تابع تبدیل فیلتر پایین‌گذر IIR از هر چهار نوع ذکر شده را با مشخصات زیر بدست آورد:

نرخ نمونه برداری 40kHz، فرکانس مرز باند عبور 4kHz، فرکانس مرز باند توقف 8kHz، ریبیل باند عبور 0.5dB و حداقل میرایی باند توقف 40dB.

بخش ۲: طراحی فیلتر IIR

بعد از انتخاب نوع فیلتر و تعیین درجه‌ی آن، باید تابع تبدیل فیلتر را محاسبه کنیم. توابع زیر را در متلب به منظور محاسبه‌ی تابع تبدیل با درجه مطلوب استفاده می‌کنند.



$$[num, then] = butter(N, W_n)$$

$$[num, then] = butter(N, W_n, 'high')$$

$$[num, then] = butter(N, W_n, 'stop')$$

$$[num, then] = cheby1(N, R_p, W_n)$$

$$[num, then] = cheby1(N, R_p, W_n, 'filtertype')$$

$$[num, then] = cheby2(N, R_s, W_n)$$

$$[num, then] = cheby2(N, R_s, W_n, 'filtertype')$$

$$[num, then] = ellip(N, R_p, R_s, W_n)$$

$$[num, then] = ellip(N, R_p, R_s, W_n, 'filtertype')$$

فعالیت ۳: پارامترها و عملکرد هر یک از توابع بالا را توضیح دهید.

فعالیت ۴: برنامه‌ای بنویسید که یک فیلتر بالاگذر چبی شف نوع ۲ را با مشخصات زیر طراحی کند:

$$f_s = 3.5 \text{ kHz}$$

$$\omega_p = 1050 \text{ Hz}$$

$$\omega_s = 600 \text{ Hz}$$

$$R_p = 1 \text{ dB}$$

$$R_s = 50 \text{ dB}$$

آزمایش ۷: نمونه برداری از سیگنال پیوسته، افزایش و کاهش نرخ نمونه برداری

در دنیای واقعی عموماً سیگنال‌هایی که وجود دارند (مانند سیگنال صوتی) سیگنال‌های پیوسته هستند. اما سیستم‌های پردازش سیگنال دیجیتال، یا کامپیوترها نیازمند داده‌های گسسته هستند. بدین ترتیب، نمونه برداری از سیگنال‌های پیوسته انجام می‌شود تا پردازش لازم در حالت گسسته بر روی آن‌ها اعمال شود و سپس تبدیل به حالت پیوسته شود. نمونه برداری از تابع پیوسته با استفاده از یک سیگنال متناوب انجام می‌شود. در این آزمایش اثر نمونه برداری را در حوزه‌ی زمان و حوزه‌ی فرکانس بر روی سیگنال مشاهده خواهیم کرد. همچنین روش‌های بازسازی سیگنال پیوسته از روی نمونه‌های گسسته آن ضمن حفظ اطلاعات سیگنال بررسی می‌شود.

در مبحث پردازش سیگنال گسسته، گاه نیاز است که نرخ نمونه برداری را توسط یک ضریب افزایش یا کاهش دهیم. عمل افزایش نرخ نمونه برداری *upsampling* و عمل کاهش آن *Downsampling* نام دارد و در این آزمایش با ویژگی‌های این دو عملگر و تاثیر آن‌ها بر سیگنال در حوزه زمان و حوزه فرکانس آشنا خواهیم شد.

Signal processing toolbox:

جعبه ابزار پردازش سیگنال:

freqs	freqz	sinc	hamming	Fir2	resample	sinc
-------	-------	------	---------	------	----------	------

بخش اول: نمونه برداری و بررسی اثرات

فعالیت ۱: یک سیگنال سینوسی با فرکانس ۱ هرتز برای مدت سه ثانیه و با تعداد نمونه‌های زیاد، مثلاً ۱۰۰ نمونه در یک ثانیه ایجاد کنید. سپس به ازای نرخ‌های نمونه برداری مختلف، نمونه برداری را انجام داده و نتیجه را گزارش کنید.

فعالیت ۲: برای دو فرکانس دیگر از سیگنال سینوسی با مقادیر ۳Hz و ۵Hz نمونه برداری را انجام دهید. چه تفاوتی میان سیگنال‌های نمونه برداری شده‌ی جدید است؟ توضیح دهید.

پس از نمونه برداری از سیگنال با نرخ‌های متفاوت، سیگنال پیوسته را از نمونه‌های آن بازسازی می‌کنیم. در صورتیکه نرخ نمونه برداری کمتر از نرخ نایکویست باشد، سیگنال به طور صحیح بازسازی نخواهد شد و دچار تخریب خواهد شد. که به آن *aliasing* می‌گویند.

فعالیت ۳: سیگنال فعالیت ۱ را با فرکانسهای مختلف نمونه برداری کنید. حد نایکویست را برای آن تعیین کنید. سپس فرکانس سیگنال ورودی را به مقادیر 3Hz و 5Hz تغییر داده و خروجی را مشاهده کنید. آیا سیگنال بازسازی شده تغییر کرده است؟ چرا؟

فعالیت ۴: یک سیگنال کسینوسی با فرکانسهای ۲۵۰۰ هرتز و ۴۵۰۰ هرتز زیر را با فرکانس ۸۰۰۰ هرتز نمونه برداری کنید.

$$x(t) = 5 \cos(2\pi \cdot 2500t) + 2 \cos(2\pi \cdot 4500t), \text{ for } t \geq 0,$$

الف) طیف حوزه فرکانس سیگنال را تا 20kHz در نرم افزار متلب نمایش دهید.

ب) فرض کنید که از یک فیلتر باتروث آنتی الیاسینگ مرتبه ۲ با فرکانس قطع ۴۰۰۰ هرتز قبل از نمونه برداری استفاده شده باشد، ابتدا نمودار طیف حوزه فرکانس را مجددا ترسیم کنید. سپس شکل موج آنالوگ بازیابی شده سیگنال را در حوزه زمان ترسیم نمایید.

بخش ۲: بررسی افزایش نرخ نمونه برداری (upsampling) و کاهش نرخ نمونه برداری (Downsampling)

فعالیت ۵: افزایش نرخ نمونه برداری به صورت زیر انجام می شود. الف) به ازای دو مقدار دلخواه از L ، رابطه ی بین سیگنال ورودی و خروجی را تحقیق کنید. ب) به ازای مقادیر $L = 2, L = 3, L = 5$ اجرا کرده و خروجی را مشاهده کنید. چه نتیجه ای می گیرید؟ آیا می توانید رابطه ای بین طیف ورودی و خروجی را بیابید؟

$x[n] \rightarrow \boxed{\uparrow L} \rightarrow x_u[n]$

$$x_u[n] = \begin{cases} x[n/L], & n = 0, \pm L, \pm 2L, \dots, \\ 0, & \text{otherwise.} \end{cases}$$

فعالیت ۶: کاهش نرخ نمونه برداری به صورت زیر انجام می شود. الف) عملیات کاهش نرخ را به ازای دو مقدار مختلف M انجام داده و خروجی را مشاهده کنید. چه نتیجه ای می گیرید. ب) به ازای مقادیر $M = 2, M =$

$M = 5$, اجرا کرده و خروجی را مشاهده کنید. چه نتیجه‌ای می‌گیرید؟ آیا می‌توانید رابطه‌ای بین طیف ورودی و خروجی را بیابید؟

$$x[n] \longrightarrow \boxed{\downarrow M} \longrightarrow y[n] \quad y[n] = x[nM].$$

آزمایش ۸: پردازش تصاویر دیجیتال

هر تصویر دیجیتال در متلب، آرایه‌ای دو بعدی و یا سه بعدی (برای تصاویر رنگی) از اعداد است. در این آزمایش با خواندن تصاویر دیجیتال در متلب، عملیات‌های ابتدایی پردازش بر روی تصاویر، ترکیب و ذخیره تصاویر آشنا خواهیم شد.

دستورات متلب مورد نیاز:

image processing toolbox:

جعبه ابزار پردازش تصویر :

imread	imwrite	imfinfo	imnoise	Mat2gray	Rgb2gray	Im2uint8
im2uint16	imhist					

image processing toolbox:

جعبه ابزار پردازش تصویر :

imread	imwrite	imfinfo	imnoise	Mat2gray	Rgb2gray	Im2uint8
im2uint16	imhist	imfilter	histeq	Filter2	Medfilt2	fspecial

بخش ۱: خواندن تصویر و مشاهده هیستوگرام

فعالیت ۱: الف) با استفاده از دستور imread، تصویر موجود در فولدر آزمایش به نام image1.tif را فراخوانده و در متغیر a ذخیره کنید. ابعاد آرایه تصویر چند است و نوع داده‌های آن چیست؟ ب) تصویر a را با استفاده از دستور imshow مشاهده و سپس هیستوگرام آن را با استفاده از دستور imhist رسم کنید.

فعالیت ۲: فعالیت ۱ را برای تصویر image2 نیز انجام دهید. هیستوگرام این تصویر چه تفاوتی با هیستوگرام تصویر قبل دارد؟ با توجه به تعریف هیستوگرام، این تفاوت را چگونه توجیه میکنید؟

فعالیت ۳: الف) آیا میتوانید تصویر image2 را واضح‌تر کنید؟ (سعی کنید با استفاده از تابع معادله خط هیستوگرام را پخش کنید). ب) حال دستور histeq را بر روی تصویر اعمال کرده و خروجی و هیستوگرام آن را ترسیم کنید.

فعالیت ۴: تصویر image3 را خوانده و در متغیر a ذخیره کنید. تصویر را مشاهده کنید. پیکسل‌های مربوط به کانال‌های R و G و B را بصورت جداگانه در سه متغیر ذخیره کرده و نمایش دهید. هیستوگرام مربوط به هر رنگ را نیز نمایش دهید.

بخش ۲: اعمال تبدیل روی تصاویر

فعالیت ۵: تصویر image1.tif را فراخوانی کرده و ابعاد آن را به مقدار 100*100 تغییر دهید و تصویر جدید را مشاهده کنید. هیستوگرام تصویر جدید را با تصویر قبل مقایسه کنید. آیا تفاوتی مشاهده می‌شود؟ چرا؟

فعالیت ۶: با استفاده از دستورات مناسب، تصویر image1 را ۳۰ درجه و ۴۵ درجه و ۹۰ درجه و ۱۸۰ درجه بچرخانید و در کنار هم ترسیم کنید.

فعالیت ۷: تصویر image4 را خوانده و سپس با استفاده از آستانه‌گذاری بصورت سیاه و سفید تبدیل کنید. سپس با استفاده از دستورات سایش و گسترش مورد نیاز تدايره‌ها را تفکیک کرده و خطوط و نویزها را حذف کنید. سپس با استفاده از دستورات مناسب connectedcomponents تعداد دایره‌ها را شمارش کنید.

بخش ۳: اعمال فیلترهای مختلف بر روی تصاویر

به منظور حذف نویزهای با فرکانس مکانی بالا از فیلتر پایین‌گذر استفاده می‌کنیم تا تصویر اصطلاحاً نرم‌تر شود. فیلتر کردن تصویر به این صورت است که پنجره‌های با ابعاد معین بر روی تصویر حرکت کرده و روی پیکسل‌ها تاثیر می‌گذارد. ساده‌ترین فیلتر که برای حذف نویز استفاده می‌شود، فیلتر میانگین است. در این نوع فیلتر به جای هر پیکسل در تصویر، میانگین پیکسل‌های همسایگی آن قرار داده می‌شود. بعنوان مثال، عملکرد این فیلتر را برای پنجره‌ی با ابعاد سه در سه در شکل زیر مشاهده می‌کنید.

	a	b	c
	d	e	f
	g	h	i

$$\rightarrow \frac{1}{9}(a + b + c + d + e + f + g + h + i)$$

فعالیت ۸: الف) برنامه‌ای بنویسید که برای تصویر image6 فیلتر میانگین با ابعاد 3*3 را اعمال کند. در این تمرین از توابع آماده متلب استفاده نکنید و برنامه را با استفاده از حلقه خودتان بنویسید. ب) سایز فیلتر را تغییر دهید ۵*۵ و ۷*۷ و ۱۱*۱۱ چه نتیجه‌ای می‌گیرید؟

فعالیت ۹: به تصویر image7 نویز نمک فلفل با درصدهای مختلف اضافه کنید. سپس با استفاده از تابع fspecial و دستور filter2 انواع فیلترهای مختلف را بر روی آن اعمال کنید و نتیجه را مقایسه کنید.

نوع	توضیحات
average	فیلتر میانگین
disk	فیلتر میانگین دایروی
gaussian	فیلتر گوسی پایین گذر
laplacian	تقریب دوبعدی عملگر لاپلاسی
log	لاپلاس فیلتر گوسی
motion	تقریب حرکت خطی دوربین
prewitt	مشخص کردن لبه
sobel	مشخص کردن لبه

فعالیت ۱۰: لبه یابی (فیلتر بالاگذر)، به منظور تعیین لبه‌ها می‌توانی با استفاده از روابط مشتق اول و دوم بخش‌های فرکانس بالای تصویر (یعنی جاهایی که رنگ یا شدن روشنایی تغییر ناگهانی پیدا میکند) را پیدا کنیم. برای این منظور در متلب می‌توانیم از فیلترهای زیر استفاده کنیم. انواع این فیلترها را بر روی تصویر image7 اعمال کنید و نتیجه را بررسی کنید.

	1. Prewitt operator	2. Sobel operator
vertical	$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$
horizontal	$\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$	$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$

آزمایش ۹: پردازش سیگنال‌های صوتی

خواندن فایل استریوی صوتی، پخش آن و تبدیل آن به مونو، و انتخاب یک بخش از فایل صوتی

الف) با دستور `[y Fs] = audioread('Homayoun.mp3');` ابتدا فایل صوتی را بخوانید.

سایز y را با دستور `size(y)` مشاهده کنید، آیا میدانید هر عدد یعنی چه؟

ب) می‌خواهیم فایل خوانده شده را در MATLAB پخش کنیم، برای این منظور از دو دستور زیر استفاده می‌کنیم

```
p = audioplayer(y,Fs);
```

```
play(p);
```

ج) برای مونو کردن فایل، چند راه داریم، فقط صدای روی کانال ۱ را برداریم، یا فقط کانال ۲ را یا نمونه‌های دیجیتالی دو کانال را با هم جمع و بر ۲ تقسیم کنیم. که از راه سوم استفاده می‌کنیم.

دستور `sum(y,2)` جمع همه مولفه‌های روی بعد دوم ماتریس را محاسبه می‌کند. بعد اول ماتریس سطرها هستند و بعد دوم ستون‌ها هستند، پس در هر سطر همه مولفه‌های روی ستونهای مختلف را با هم جمع می‌کند.

```
ymono = sum(y,2)/2;
```

```
size(ymono)
```

د) می‌خواهیم حدود ۱۰ ثانیه از فایل صوتی را از ثانیه ۳۱ تا ثانیه ۴۱ جدا کنیم و در یک فایل جدید ذخیره کنیم.

با توجه به اینکه فرکانس نمونه برداری را داریم $F_s=44100$ ، لذا میدانیم که ثانیه ۳۱، یعنی سمپل شماره 31×44100 ، لذا زمان شروع و پایان را بصورت زیر تعیین می‌کنیم.

```
t_start = 31*Fs;
```

```
t_stop = 41*Fs;
```

حال با دستور زیر بازه شروع و پایان را از سیگنال `ymono` جدا می‌کنیم و در فایل `ynew` میریزیم و سپس آن را پخش می‌کنیم.

```
ynew = ymono(t_start : t_stop);
```

```
p = audioplayer(ynew,Fs);
```



```
play(p);
```

سپس با دستور audiowrite فایل جدا شده را در کامپیوتر و فولدر کاری با فرمت wav ذخیره سازی میکنیم.

```
audiowrite('homayoon.wav', ynew, Fs)
```

ه) میخواهیم صدای پخش صدا را بالا ببریم، برای این منظور یک بهره تعریف میکنیم $A = 2$ و سپس از دستور زیر استفاده میکنیم

```
Amplifiedy = A * ynew;
```

```
p = audioplayer(Amplifiedy, Fs);
```

```
play(p);
```

بهره را زیاد کنید، چه اتفاقی می افتد؟ توضیح دهید چرا این اتفاق می افتد؟

و) در هنگام پخش فرکانس را بجای F_s برابر $2 * F_s$ قرار دهید. چه اتفاقی می افتد؟ توضیح دهید.

```
p = audioplayer(ynew, 2*Fs);
```

```
play(p);
```

فرکانس را $F_s/2$ قرار دهید. نتیجه را توضیح دهید.

۳- خواندن صدا، ضبط صدا با یک فرکانس نمونه برداری دیگر، پنهان کردن صدا و سپس بازیابی آن

در این تمرین می خواهیم ابتدا یک صدا از خودتان در مورد درس DSP ضبط کرده و سپس آن را در فایل صوتی ایجاد شده در مرحله قبل پنهان کرده و بعنوان پاسخ ارسال کنید.

الف) برای ضبط صدا از دستور `audiorecorder` استفاده میشود میتوانید به `help` متلب مراجعه کنید، اگر از این دستور به صورت `recObj = audiorecorder` استفاده شود، از مقادیر پیش فرض یعنی $F_s = 8000$ ، تعداد بیت ها 8 بیت و تعداد کانالها ۱ یعنی مونو یک شیء `recorder` درست میشود، که فعلا برای ما کفایت میکند.

سپس با استفاده از دستور `recordblocking(recObj, recDuration)` به میزان مقدار ثانیه ای که در متغیر `recDuration` تعیین کرده ایم، صدا ضبط میشود.

برای نشان دادن لحظه شروع به ضبط و لحظه پایان ضبط میتوانیم از دستور `display` بران اعلان روی صفحه استفاده کنیم.

حداکثر ۱۰ ثانیه صدا در از خودتان در مورد درس DSP ضبط کنید. در کد زیر زمان ضبط ۵ ثانیه تعیین شده.

```
recDuration = 5; % record for 5 seconds
disp('Start speaking.');
```

```
recordblocking(recObj, recDuration);
```

```
disp('End of recording. Playing back ...');
```

در مرحله بعد باید صدا را در یک متغیر ریخته و سپس آن را ذخیره کنیم. برای گرفتن صدا و ریختن در یک متغیر از دستور `getaudiodata(recObj)` استفاده میکنیم.

```
my_voice = getaudiodata(recObj);
```

ب) حال صدای ضبط شده را پخش کنید. در تمرین قبل توضیح داده شده است. دقت کنید که $F_s=8000$ است.

```
Fs = 8000;
p = audioplayer(my_voice, fs);
play(p);
```

ج) صدا را با نام `My_DSP_voice` و با فرمت `wav`. ذخیره کنید. در تمرین قبل در مورد ذخیره یک صدا توضیح داده شده است.

دقت کنید که $F_s=8000$ است.

د) در مرحله بعد میخواهیم این صدای ضبط شده را در داخل صدای ۱۰ ثانیه ای ذخیره شده در تمرین دوم پنهان کنیم. ساده ترین کار جمع دو صدا است. برای پنهان شدن بیشتر میتوانید تضعیف شده صدای ضبط شده را پنهان کنید. دقت کنید چون فرکانس نمونه برداری ها متفاوت است اگر ۵ ثانیه صدا با فرکانس ۸ کیلو ضبط کرده باشید، معادل ۱ ثانیه صدا در فرکانس ۴۰ کیلو خواهد بود، یعنی کل سیگنال صدای شما در ثانیه اول سیگنال آهنگ مثال ۲ قرار خواهد گرفت.

یک فایل m.file دیگر با نام S1_Exc3_part2 ایجاد کنید و دو صدای ذخیره شده را به همراه فرکانس های نمونه بردارشان بخوانید.

```
[y1,Fs1] = audioread('homayoon.wav');  
[y2,Fs2] = audioread('My_DSP_voice.wav');
```

برای جمع دو بردار باید طول دو بردار یکسان باشند. در حالی که با استفاده از دستور `length(y1)` یا `numel(y2)` یا دستور `size(y1)` میتوانید طول دو سیگنال خوانده شده را مقایسه کنید، متوجه میشوید که طولشان با هم برابر نیست، پس نمیتوانید آنها را جمع کنید.

```
N1 = numel(y1)  
N2 = length(y2)
```

بنابراین N2 درایه اول بردار y1 را با y2 جمع میکنیم و در بردار sy میریزیم.

```
sy = y(1:N2)+y2;
```

یک متغیر y تعریف میکنیم و سیگنال با طول بزرگتر را در آن میریزیم. سپس N2 مولفه اول آن را با sy جایگزین میکنیم.

```
y = y1;  
y(1:N2) = sy;
```

البته میتوان هر دو مرحله را در یک خط بصورت `y(1:N2) = y(1:N2)+y2` نوشت که دیگر نیازی به تعریف متغیر sy نباشد.

صدای حاصله را با نام `watermarked_voice` ذخیر کنید.

```
audiowrite('watermarked_voice.wav', y, Fs1);
```

ه) حال می‌خواهیم برنامه ای برای استاد بنویسیم که صدای ما را استخراج کند. کلید همان ۱۰ ثانیه صدای آهنگ است. m فایل جدیدی به نام S1_Exc3_part3 ایجاد کنید.

ابتدا دو فایل واترمارک شده و کلید را بخوانید.

```
[y_key,Fs1] = audioread('homayoon.wav');  
[y_watermarked,Fs2] = audioread('watermarked_voice.wav');
```

سپس آنها را از هم کم کنید

```
y = y_watermarked - y_key;
```

و نتیجه را play کنید.

برای play شدن صحیح صدای استخراج شده باید چه کاری انجام دهید؟

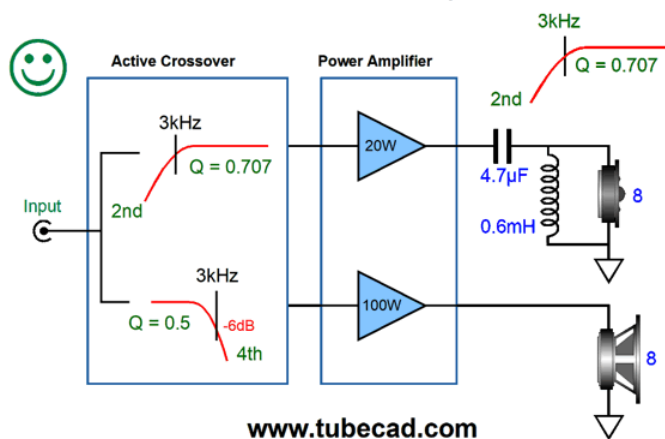
۴- سیستم crossover صوتی دیجیتال

همانطور که میدانید فرکانس شنوایی انسان 20Hz-20KHz است. اما اسپیکرها بدلیل محدودیت های مکانیکی قابلیت پوشش همه این محدوده را ندارند. لذا از دو بلندگو یکی برای پوشش صداهای بم و یکی برای پوشش صداهای زیر استفاده میشود. اما سیستم پردازش صدا باید ابتدا محدوده این دو فرکانس را جدا کند و سپس به بلندگوهای مربوطه ارسال کند، به این سیستم، سیستم crossover میگویند. در واقع

سیستم crossover: سیگنال صوتی را به باندهای فرکانسی مختلف می شکند و هر محدوده را برای اسپیکر مربوطه ارسال میکند. در این تمرین میخواهیم بخش پردازش سیگنال این سیستم را پیاده سازی کنیم.



Improved Bi-Amp Setup with Linkwitz-Riley 4th-Order Crossover



برای انجام این تمرین مراحل زیر را انجام دهید:

الف) با استفاده از دستور `[y,Fs] = audioread('filename.format')` صدای مد نظر را بخوانید. به جای filename.format نام فایل صوتی مدنظرتان را قرار دهید. که در اینصورت sample های فایل در y و فرکانس نمونه برداری در Fs قرار میگیرد.

ب) با استفاده از دستور $Xf = \text{fft}(y)$ سیگنال را به حوزه فرکانس برده و در متغیر Xf قرار دهید.

ج) میدانید که دستور fft تبدیل فوریه سیگنال را در بازه $[0 \ 2\pi]$ نشان میدهد و ما علاقه مندیم تبدیل فوریه را در بازه $[-\pi \ \pi]$ مشاهده کنیم، لذا از دستور $\text{fftshift}(Xf)$ برای این منظور استفاده کنید و مقدار آن را در متغیر با نام دلخواهتان مثلاً XfS بریزید.

د) حال میخواهیم هم fft سیگنال و هم شیفต์ خورده آن را ملاحظه کنیم. برای این منظور باید محور x که همان فرکانس است را درست کنیم. میدانیم که تعداد مولفه های فرکانسی به تعداد سمپل ها است. تعداد سمپل ها N را میتوان با اندازه گیری طول y بدست آورد: $N = \text{length}(y)$ ، و میدانیم که رزولوشن فرکانس برابر است با Fs/N که با $f0$ نمایش میدهیم. بنابراین محور فرکانس $f = 0 : f0 : (N - 1)f0$ خواهد بود. و برای نمایش شیف یافته محور فرکانس (x) برابر $f = -\left(\frac{N}{2}\right)f0 : f0 : \left(\frac{N-1}{2}\right)f0$ خواهد بود. برای ترسیم در هر حالت نیز میتوانید از دستور plot مثلاً به صورت $\text{plot}(f, \text{abs}(Xf))$ استفاده کنید. دلیل استفاده از abs این است که Xf یک عدد مختلط است یعنی حاوی دامنه و فرکانس است که فعلاً ما با فاز آن کاری نداریم و میخواهیم دامنه را ترسیم کنیم. نکته بعدی اینکه اگر چند plot پشت سر هم استفاده کنید، plot قبلی پاک میشود و آخرین plot نمایش داده میشود. برای اینکه چنین اتفاقی نیافتد و هر تصویر بصورت جداگانه ترسیم شود قبل از plot از یک دستور figure استفاده کنید تا plot جدید در یک figure با شماره جدید ترسیم شود.

ه) حال اگر بخواهید محدوده فرکانسی را فیلتر کنید باید پاسخ فرکانسی شیف یافته را در فیلتر پایین گذر ضرب کنید. ساده ترین راه برای ایجاد فیلتر پایین گذر دیجیتال ایده آل، استفاده از دستور trapmf است. (این دستور دوزنقه تولید میکند) برای این منظور ابتدا فرکانس قطعی که مدنظرتان است را بنویسید مثلاً $fc = 3000$ و سپس فیلتر دیجیتال را بصورت زیر تعریف کنید: $\text{LPFilt} = \text{trapmf}(f, [-fc, -fc, fc, fc])'$ (دقت کنید علامت colon در انتها برای ترنسپوز کردن ماتریس الزامی است یعنی بردار ستونی را به سطری تبدیل میکند). فیلتر را با دستور $\text{figure; plot}(f, \text{LPFilt}, 'r')$ ترسیم کنید. و نتیجه را ببینید.

حال فیلتر را در سیگنال ضرب نقطه ای کنید. مثلاً $XfS_{\text{Filtered}} = XfS .* \text{LPFilt}$

و) برای اینکه اثر اعمال فیلتر را بر روی سیگنال صوتی ببینید، باید سیگنال فیلتر شده را مجدداً به حوزه زمان ببرید. بنابراین ابتدا از دستور fftshift باید استفاده کنید و سپس از رابطه ifft عکس تبدیل فوریه را محاسبه کنید.

برای مثال اگر نام متغیرهایتان تا اینجا با متن یکی باشد $\text{LPFilterdSignal} = \text{ifft}(\text{fftshift}(XfS_{\text{Filtered}}))$

سپس با استفاده از دو دستور زیر میتوانید سیگنال صوتی را پخش کنید.

```
p = audioplayer(real(LPFilterdSignal),Fs);
```

```
play(p);
```

دقت کنید چون ما در سیگنال تغییراتی ایجاد کرده ایم، عکس تبدیل فوریه سیگنال یک عدد مختلط است که ما فقط به قسمت حقیقی آن نیاز داریم لذا برای پخش سیگنال صوتی از دستور `real(LPFilterdSignal)` استفاده کرده ایم. میتوانید مقدار `fc` را در برنامه تغییر دهید و نتیجه را گوش کنید.

ز) برای اعمال فیلتر بالاگذر ابتدا باید آن را بسازید، ساده ترین راه تفریق فیلتر پایین گذر ساخته شده در مرحله ه از عدد ۱ است. $HPFilt = 1 - LPFilt$ را نوشته و سپس با دستور `figure; plot(f,HPFilt,'r')` پاسخ فرکانسی فیلتر جدید را ترسیم کنید. حال مجدد فیلتر جدید را در تبدیل فوریه شیف خورده ضرب کنید. سپس عکس تبدیل فوریه را مشابه مرحله واو محاسبه کنید نام آن را `HPFilterdSignal` و سیگنال حاصله را پخش کنید. با فرکانس قطع بازی کنید و نتیجه را گوش کنید.

توجه: در هنگام اجرای برنامه فقط آخرین صدا پخش میشود، یا از بولت باید استفاده کنید، و یا از دستور `pause(10)` قبل بصورت `play` از دستور `pause(10)` استفاده کنید تا هم صدای فرمانس بالا و هم فرمانس پایی را بتوانید گوش بدهید.

ح) میخواهیم دو صدا را همزمان بصورت استریو پخش کنیم یعنی فرکانس بالا را از بلندگوی راست و فرکانس پایین را از بلندگوی چپ پخش کنیم. پس باید سیگنال را دو کاناله کنیم :

```
Signalsterio = [HPFilterdSignal, LPFilterdSignal];
```

حال این سیگنال را پخش کنید.

```
p = audioplayer(real(Signalsterio),Fs);
```

```
play(p);
```

آزمایش ۱۰: پردازش سیگنال‌های تصادفی EEG با هوش مصنوعی

فعالیت ۱: برنامه ای بنویسید که سیگنال‌ها حوزه زمان یک فرد سالم و یک فرد بیمار و یک فرد در حال تشنج را در یک تصویر نمایش دهد. (از دستور subplot(3,1,n استفاده کنید).

فعالیت ۲: برنامه ای بنویسید که تبدیل از سیگنال همان افراد سالم و بیمار و در حال تشنج قبلی تبدیل فوریه بگیرد و در یک تصویر با استفاده از دستور subplot نمایش دهید.

فعالیت ۳: رزولوشن فرکانسی را مشخص کنید و سیگنال‌ها را بر حسب فرکانس واقعی ترسیم کنید. آیا نویز برق شهر را میتوانید تشخیص دهید؟

فعالیت ۴: برنامه ای بنویسید که با استفاده از یک فیلتر باترورث میان نگذر، فرکانس برق شهر را حذف کنید.

فعالیت ۵: برنامه ای برای جدا کردن محدوده های فرکانسی ۰-۴ و ۴-۸ و ۸-۱۲ و ۱۲-۳۰ و ۳۰ به بعد بنویسید. سپس سیگنالهای جدا شده را به حوزه زمان برده و در یک تصویر با استفاده از subplot ترسیم کنید.

فعالیت ۶: تابعی برای استخراج ویژگی های میانگین، واریانس، چولگی، تیزی، انتروپی و انرژی از یک سیگنال بنویسید.

فعالیت ۷: از سیگنال های باندهای فرکانسی مختلف در حوزه زمان ویژگی های فوق را استخراج کنید. و یک بردار ویژگی با طول ۵*۶ (۶ ویژگی از ۵ باند فرکانسی) برای هر فرد استخراج کنید.

فعالیت ۸: با استفاده از روش knn و روش mlp با در نظر گرفتن ۷۰ درصد داده ها به عنوان داده های آموزش و ۳۰ درصد به عنوان داده های تست، دقت بدست آمده روی داده های آموزش و داده های تست را برای دو حالت زیر اندازه گیری و گزارش کنید. (ضریب k در روش knn و لایه های و تعداد نورن در هر لایه را در روش mlp تغییر دهید، ببینید کدام حالت جواب بهتری میدهد)

الف- سالم و بیمار

کدهایی که ممکن است مورد استفاده قرار دهید:

```
load eegdata

sig1= A(:,1);
sig2= E(:,1);

%% fourier transform
N= length(sig);
fx1= fft(sig)';
fx1= abs(fx1(1:round(N/2)));
%% step 4: calculate frequency resolution
rf= linspace(0,fs/2,round(N/2));
figure
stem(rf,fx1,'b','linewidth',1,'marker','none')

% stop butterworth filter
order=3;
fl= 49;
fh= 51;
wn=[fl fh] / (fs/2);
type= 'stop';
[b,a]= butter(order,wn,type);
sig_dn= filtfilt(b,a,sig);

function [features] = myfeatureExtraction(sig)
    mu= mean(sig);
    v= var(sig);
    sk= skewness(sig);
    kr= kurtosis(sig);
    H= wentropy(sig,'shannon');
    P= mean(sig.^2);
    features= [mu;v;sk;kr;H;P];
end
```

```

band= [0.1,4,8,12,30;
       4,8,12,30,70];
for j= 1:size(band,2)
    % design bandpass filter
    wn= band(:,j)/ (fs/2);
    [b2,a2]= butter(3,wn,'bandpass');
    % apply designed bandpass filter
    sigA1= filtfilt(b2,a2,sigA);

```

```

%% step 5: determine coefficients of delta rhythms
band= [0.1,4,8,12,30;
       4,8,12,30,70];
name= {'Delta','Theta','Alpha','Beta','Gamma'};
for j=1:size(band,2)
    fl= band(1,j);
    fh= band(2,j);
    indx= find(rf>= fl & rf<=fh);
    stem(rf(indx),fx(indx),'linewidth',1,'marker','none')
    xpos=rf(indx(1));
    ypos= max(fx(indx));
    text(xpos,ypos,name{j},'fontsize',20)
    drawnow
end

```

```

%% step 2: train model using train data and train label
mdl= fitcknn(traindata',trainlabel,'NumNeighbors',3);
%% step 3: test trained model using test data
output= predict(mdl,testdata');

```

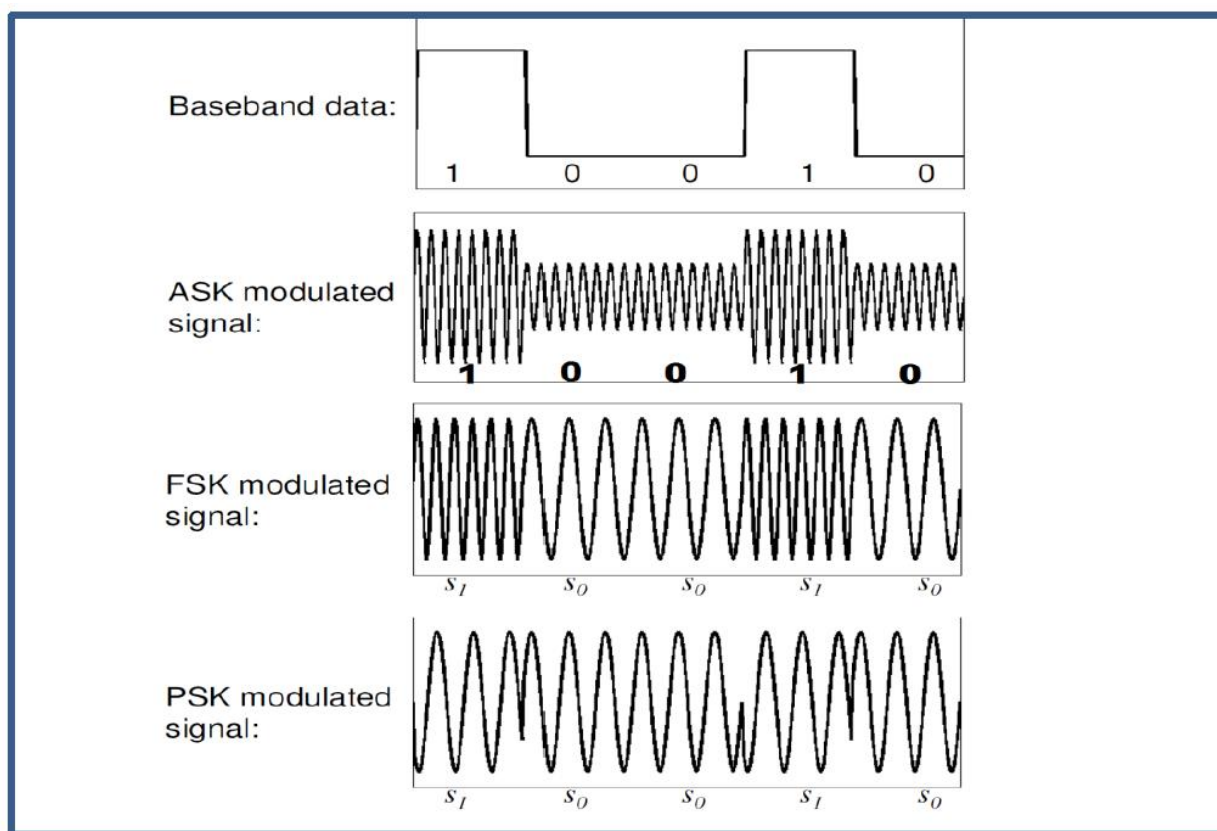
```

%% step 4: validation
accuracy= sum(output==testlabel) / numel(testlabel) *100;

```

آزمایش ۱۱: پیاده‌سازی برخی مدولاسیون‌های مخابرات دیجیتال

امروزه بسیاری از سیستم‌های فرستنده و گیرنده از سیگنال‌های دیجیتال استفاده می‌کنند. ذخیره‌سازی سیگنال‌ها در رایانه‌ها بصورت دیجیتال صورت گرفته و پردازش دیجیتال بر روی آن‌ها انجام می‌شود. همچنین در بحث مخابرات، به منظور ارسال دیتای دیجیتال در کانال مخابراتی، از مدولاسیون دیجیتال استفاده می‌شود. مدولاسیون دیجیتال به این معنی است که ویژگی‌های سیگنال حامل با توجه به بیت‌های ورودی تغییر می‌کند. این تغییرات می‌تواند در دامنه، فرکانس و یا فاز سیگنال حامل اعمال شود. در این آزمایش با نحوه ساخت تعدادی از این مدولاسیون‌ها و خواص آن‌ها آشنا می‌شویم.



بخش ۱: مدولاسیون ASK

در این مدولاسیون، تغییرات داده ورودی بر روی دامنه سیگنال حامل اعمال می‌شود. به این ترتیب که به ازای هر سطح از ورودی، دامنه‌ای متفاوت از سیگنال حامل استفاده می‌شود. در این آزمایش با این نوع از مدولاسیون آشنا شده و آن را بر روی داده ورودی دلخواه اعمال می‌کنیم.

فعالیت ۱: برنامه‌ای برای مدولاسیون ASK بنویسید. فرکانس حامل را ۲۰ و فرکانس پالس ورودی را ۴ قرار دهید. سپس شکل موج خروجی را جهت انتقال دیتای ۱۰۰۱۰۱۱۰ نمایش دهید.

فعالیت ۲: برنامه را به ازای فرکانس‌های مختلفی از سیگنال حامل اجرا کنید.

فعالیت ۳: آیا می‌توانید برنامه‌ای برای آشکارسازی مدولاسیون ASK بنویسید؟

بخش ۲: مدولاسیون BPSK

در این نوع مدولاسیون که با نام 2PSK نیز شناخته می‌شود، با تغییر داده ورودی، فاز سیگنال حامل ارسالی عوض می‌شود. حالت BPSK حالت ساده از مدولاسیون PSK است که با تغییر داده، فاز حامل به اندازه ۱۸۰ درجه جابجا می‌شود.

فعالیت ۴: برنامه‌ای برای اعمال مدولاسیون BPSK بنویسید، فرکانس حامل را ۲۰ و فرکانس پالس ورودی را ۴ قرار دهید. سپس شکل موج خروجی را جهت انتقال دیتای ۱۰۰۱۰۱۱۰ نمایش دهید.

فعالیت ۵: آیا می‌توانید برنامه‌ای برای آشکار ساز مدولاسیون BPSK بنویسید؟

بخش ۳: مدولاسیون FSK

در این نوع از مدولاسیون، تغییرات بیت‌های ورودی در فرکانس سیگنال حامل مشخص می‌شود. به گونه‌ای که به ازای بیت‌های مختلف ورودی، فرکانس‌های مختلفی از سیگنال حامل ارسال می‌شود. در حالت ساده‌ی آن مدولاسیون BFSK را خواهیم داشت که در ازای دو بیت مختلف 0,1 سیگنال حامل دو فرکانس مجزا خواهد داشت.

فعالیت ۶: برنامه‌ای برای اعمال مدولاسیون BFSK بنویسید، فرکانس حامل را ۱۰ و ۲۰ هرتز و فرکانس پالس ورودی را ۵ هرتز قرار دهید. سپس شکل موج خروجی را جهت انتقال دیتای ۱۰۰۱۰۱۱۰ نمایش دهید.

```
function [ s ] = my_FSK( x,A,f1,f2 )
```

فعالیت ۷: برنامه‌ای برای آشکارسازی مدولاسیون BFSK بنویسید.

فعالیت ۸: به سیگنال آنالوگ ارسالی در هر سه مدولاسیون فوق‌الذکر نویز اضافه کنید، سپس سعی کنید که آشکارسازی انجام بدهید. کدام نوع مدولاسیون مقاومت بیشتری نسبت به نویز نشان می‌دهد؟ توضیح دهید.